UPLIFT MODELING FOR PERSONALIZED MARKETING IN TELECOMMUNICATIONS

Applied Machine Learning, final project

Abilkhay Askar, Kolzhabay Olzhas

Group Big Data Analysis 2406, Astana IT University

CONTENTS

01	**Introduction**	Problem, why response models are not enough, research question	slides 3–4
02	**Data**	MegaFon dataset, quality checks, treatment and control balance	slides 5–6
03	**Methodology**	Our workflow, data pipeline, model architecture, how a customer is scored	slides 7–12
04	**Experiments / Results**	Metrics table, Qini curves, five splits, deciles, campaign effect	slides 13–18
05	**Discussion**	Why the T-learner won, overfitting, limitations	slides 19
06	**Conclusion**	Main findings and next steps	slides 20
07	**References**	Literature and tools	slides 21

Structure follows the final project requirements: Introduction, Data, Methodology, Experiments/Results, Discussion, Conclusion, References

WHY RESPONSE MODELS ARE NOT ENOUGH

Usual practice

A campaign is sent to customers with the highest predicted probability to buy. Many of them would buy without the offer, so the money is spent but sales do not grow.

Uplift modeling

We estimate how much the offer changes the probability to buy for each customer:

$$\text{uplift}(x) = P(Y=1 \mid x, T=1) - P(Y=1 \mid x, T=0)$$

This needs a randomized experiment: one group got the offer (T = 1), the other did not (T = 0).

	WITH THE OFFER: BUYS	WITH THE OFFER: DOES NOT BUY
WITHOUT THE OFFER: BUYS	**Sure things** Buy anyway. The offer is wasted.	**Sleeping dogs** The offer pushes them away. Do not contact.
WITHOUT THE OFFER: DOES NOT BUY	**Persuadables** Buy only because of the offer. Target them.	**Lost causes** Never buy. The offer is wasted.

Customer types: Radcliffe (2007); uplift definition: Gutierrez & Gérardy (2017)

RESEARCH QUESTION

Which subscribers should receive a marketing offer to get the most additional conversions, and does an uplift model rank them better than random targeting and a normal conversion model?

Objectives

1. Check that the experiment is randomized, so uplift can be estimated
2. Build at least two uplift models and compare them with two baselines
3. Evaluate with uplift metrics (Qini, AUUC, uplift@30%) on unseen data
4. Turn the best model into a list of customers to contact

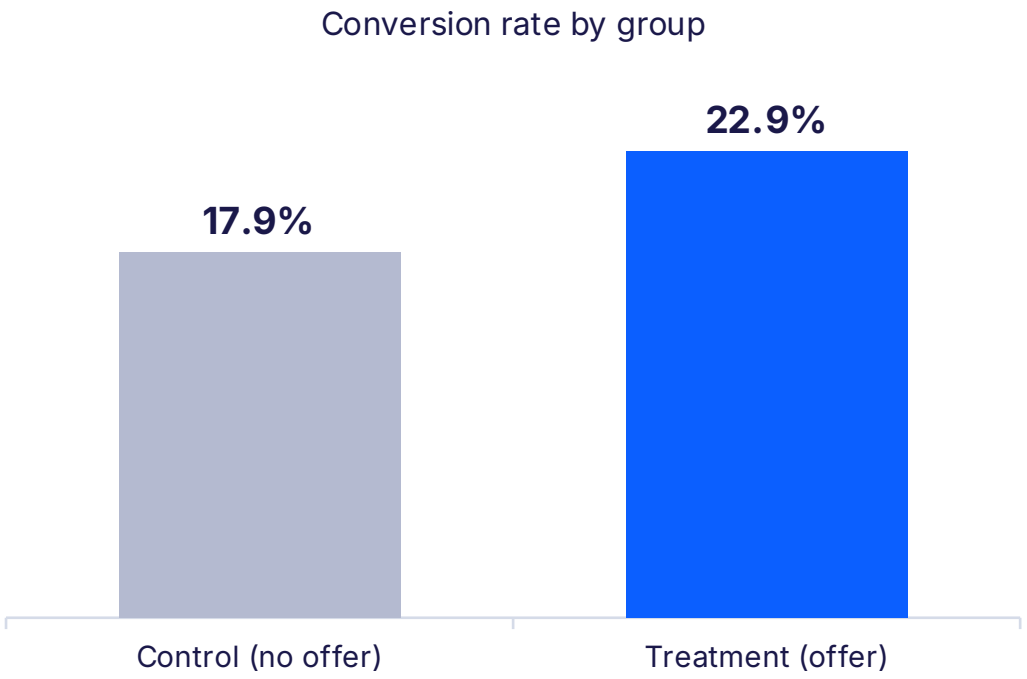
Why it matters for a telecom operator

- Every contact (SMS, call, bonus) costs money
- Operators already keep random control groups
- Millions of subscribers: a small gain in targeting gives thousands of extra conversions
- Some customers react badly to offers

THE MEGAFON DATASET

Data

PROPERTY	VALUE
Source	MegaFon Uplift Competition, ODS.ai (2021)
Type	Synthetic data that imitates a telecom campaign
Rows	600,000 labelled customers
Features	50 numeric, anonymized (X_1 to X_50)
Treatment	treatment_group: offer sent (300,368) or control (299,632)
Target	conversion, 1 if the customer converted
Missing / duplicates	0 / 0



+4.95 pp

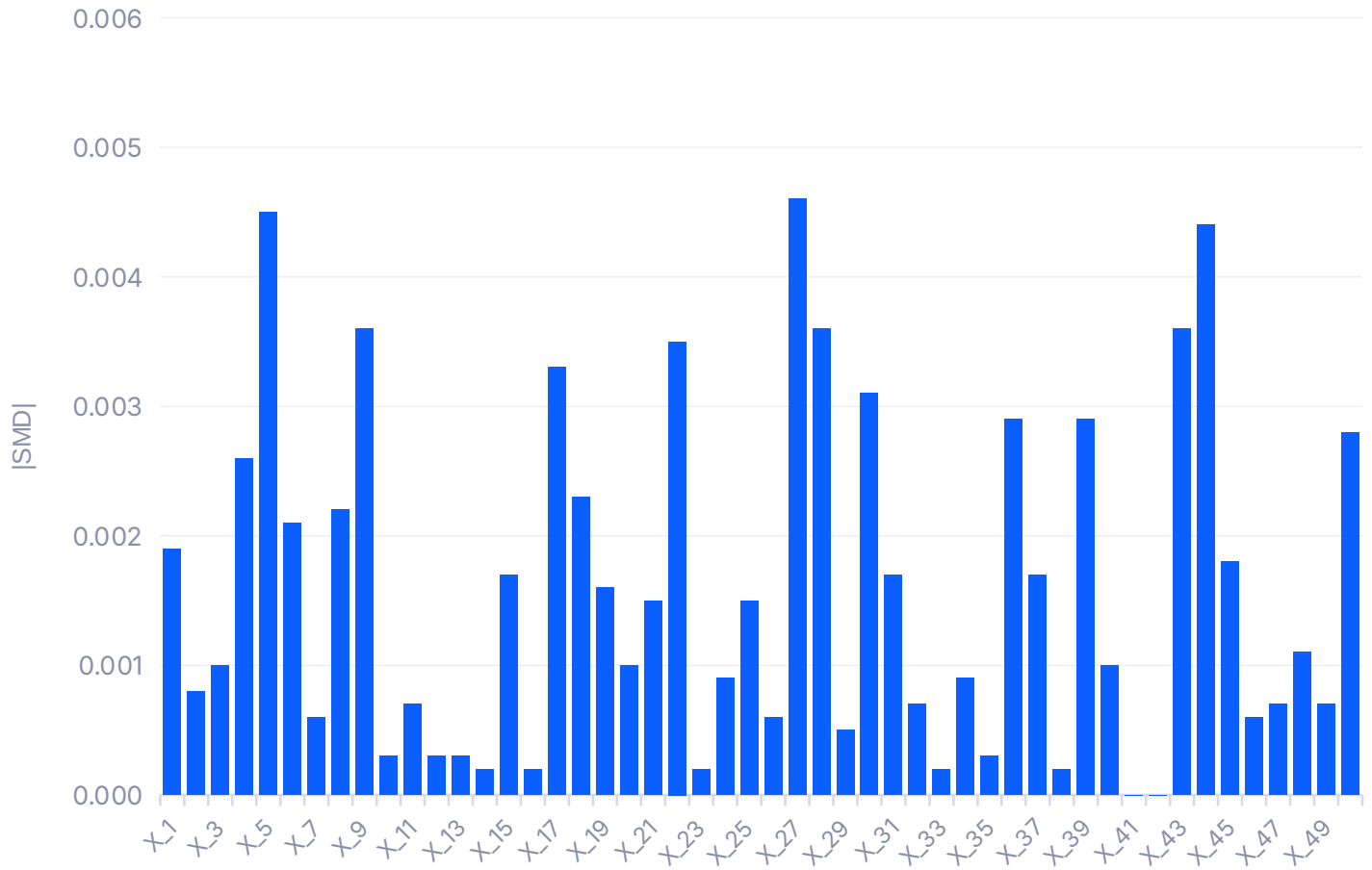
average effect of the offer (95% CI 4.75 to 5.16 pp), a 27.6% relative lift

MegaFon Uplift Competition (ODS.ai, 2021), distributed with scikit-uplift; 95% CI for the effect from a two-proportion normal approximation

DATA QUALITY AND GROUP BALANCE

Data

Standardized mean difference (SMD) between treatment and control for every feature



CHECK	RESULT
Missing values	0
Duplicate rows	0
Max |SMD|	0.0046 (X_27)
Features with |SMD| ≥ 0.1	0 of 50
Train / test rows	420,000 / 180,000
Treatment share, train / test	50.06% / 50.06%
Conversion, train / test	20.42% / 20.42%

All values are far below 0.1, so the two groups are comparable and differences in conversion come from the offer.

SMD = (mean_T − mean_C) / sqrt((var_T + var_C) / 2); usual imbalance threshold 0.1; split stratified on treatment × conversion, seed 42

OUR WORKFLOW

Methodology

How we did the project, from the proposal to the final targeting list

1 **Proposal**
Chose the dataset, metrics and 9 success criteria

2 **Data checks**
Missing values, groups, balance of 50 features

3 **Split**
70 / 30, stratified by treatment and conversion

4 **Baselines**
Random scores and a conversion classifier

5 **Uplift models**
S-learner, T-learner, class transformation

6 **Tuning**
4 settings each, 3-fold CV Qini on train

7 **Evaluation**
Test set metrics and 5 repeated splits

8 **Targeting**
Ranked list, top 30% get the offer

Tools

Google Colab, Python 3.13, pandas, scikit-learn 1.9, scikit-uplift 0.5.1, matplotlib

Rules we kept

The test set is never used for tuning. Models are chosen by Qini, not by accuracy. Every result is repeated on 5 splits.

Outputs

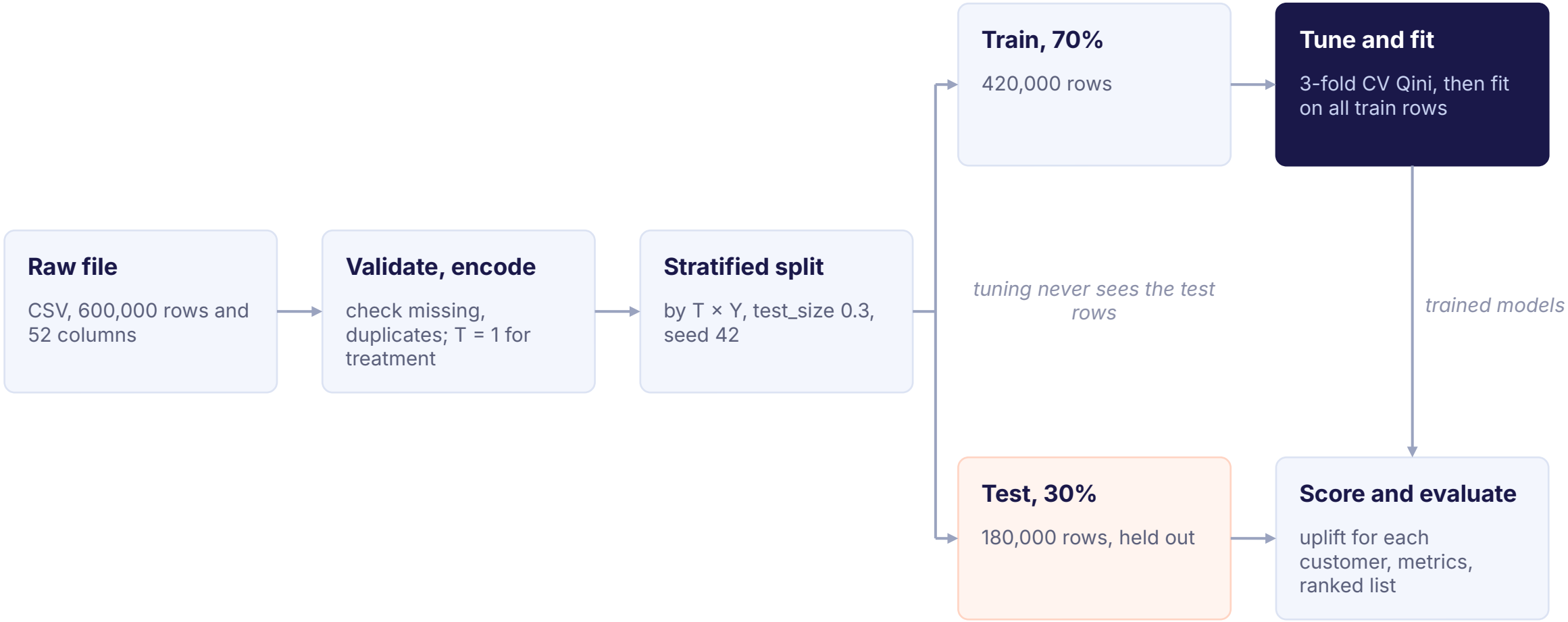
Executed notebook, figures and tables, targeting_list_top30.csv, research paper, these slides

Everything runs in one Google Colab notebook; all random seeds fixed; full run took 14 minutes on 2 CPU cores

DATA PIPELINE

Methodology

The test set is split off once and only used at the end



Notebook sections 2 to 11; stratification keeps the treatment share and conversion rate identical in both parts

MODEL ARCHITECTURE: T-LEARNER

Methodology

1. TRAINING (TRAIN SET ONLY)

Training set

420,000 customers with X, T and Y

Treated, T = 1

about 210,000 rows

Control, T = 0

about 210,000 rows

Model f_1

gradient boosting, learns P(buy | offer)

Model f_o

same settings, learns P(buy | no offer)

Each model sees only its own group, so the two can learn different patterns.

2. PREDICTION (EVERY CUSTOMER)

Customer x

50 features, treatment flag not needed

$f_1(x) = 0.90$

chance to buy with the offer

$f_o(x) = 0.07$

chance to buy without it

uplift(x) = $f_1(x) - f_o(x) = 0.83$

extra probability created by the offer

Rank all customers by uplift

top 30% get the offer, negative uplift are left out

TwoModels(method='vanilla') from scikit-uplift; base learner HistGradientBoostingClassifier: 150 trees, 63 leaves, learning rate 0.1, min_samples_leaf 100, L2 1.0

HOW THE MODEL SCORES A CUSTOMER

Methodology

Three real customers from the test set, scored by the final T-learner

CUSTOMER	P(BUY | OFFER)	P(BUY | NO OFFER)	UPLIFT	RANK	DECISION
435517	**0.90**	**0.07**	**+0.83**	**1**	**Target**
492666	0.21	0.10	+0.11	54,000	Target
519801	0.05	0.96	−0.91	180,000	Do not target

Rank 1: the offer adds 83 points of probability, the best customer to contact.

Rank 54,000: the last customer inside the 30% budget (54,000 of 180,000).

Rank 180,000: would buy without the offer (0.96) but probably not with it. A response model would contact this customer first.

notebook, sections 7 and 11

```
params = dict(lr=0.1, leaves=63,
              iters=150)
model = TwoModels(
    estimator_trmnt=hgb(**params),
    estimator_ctrl=hgb(**params),
    method="vanilla")
model.fit(X_tr, y_tr, t_tr)

uplift = model.predict(X_te)
p1 = model.trmnt_preds_  # offer
p0 = model.ctrl_preds_   # no offer
```

hgb() = HistGradientBoostingClassifier with min_samples_leaf 100, L2 1.0, seed 42

results/targeting_list_top30.csv from the notebook; probabilities rounded to two decimals

MODELS WE COMPARED

Methodology

Three uplift approaches, one linear reference and two baselines

APPROACH	HOW IT IS TRAINED	SCORE USED FOR RANKING	CLASS
S-learner	one model on X plus the flag T	f(x, 1) − f(x, 0)	SoloModel
T-learner	**two models: treated rows and control rows**	**$f_1(x) - f_o(x)$**	**TwoModels**
Class transformation	one model on Z = 1 when T equals Y	$2 \cdot P(Z=1 \mid x) - 1$	ClassTransformation
Linear T-learner	T-learner with logistic regression	$f_1(x) - f_o(x)$	TwoModels
Conversion model	one classifier on X, ignores T (baseline)	$P(Y=1 \mid x)$	scikit-learn
Random	no training (baseline)	random number	numpy

Class transformation works only because the treatment share is close to 50%, which is true here (50.06%).

Künzel et al. (2019); Jaskowski & Jaroszewicz (2012); all tree models use HistGradientBoostingClassifier (Pedregosa et al., 2011)

TRAINING SETUP AND METRICS

Methodology

Settings were chosen by 3-fold cross-validated Qini on the training set

CV QINI BY SETTING

lr / leaves / trees	S-learner	T-learner	Class transf.
0.10 / 31 / 200	0.2313	0.2364	0.2184
0.05 / 31 / 300	0.2310	0.2361	**0.2233**
0.10 / 15 / 200	0.2179	0.2292	0.2177
0.10 / 63 / 150	**0.2365**	**0.2380**	0.2179

Selected settings are highlighted. More leaves helped the S- and T-learner; class transformation preferred a lower learning rate with more trees.

Qini coefficient

Area between the model's Qini curve and the random line, divided by the area of a perfect model. Random = 0, perfect = 1. Main metric.

AUUC

The same normalization on the uplift curve.

Uplift@30%

Treatment conversion minus control conversion among the 30% of customers with the highest scores.

Why not accuracy

A model can predict who buys and still rank people badly by the effect of the offer.

Grid of 4 settings per learner (learning rate / max leaves / trees); fixed: min_samples_leaf 100, L2 1.0, no early stopping, seed 42

RESULTS ON THE TEST SET

180,000 customers that no model saw during training or tuning

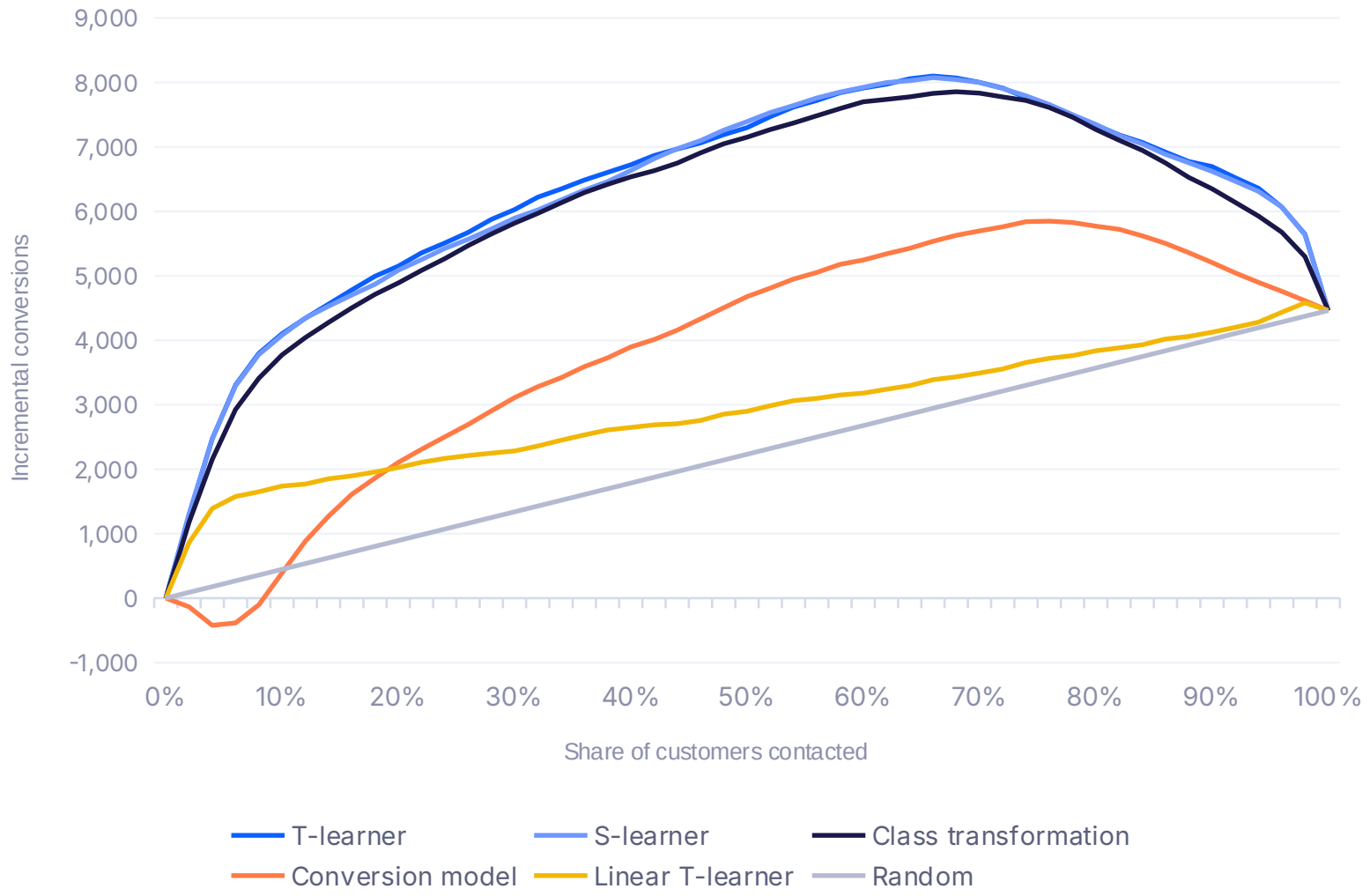
MODEL	QINI	AUUC	UPLIFT@30%	MEAN, 5 SPLITS	SD
T-learner	**0.245**	0.170	22.4%	0.244	0.0026
S-learner	**0.243**	0.168	21.9%	0.241	0.0046
Class transformation	**0.231**	0.160	21.7%	0.230	0.0027
Conversion model	**0.096**	0.067	11.5%	0.096	0.0026
Linear T-learner	**0.041**	0.028	8.5%	0.036	0.0030
Random	**−0.003**	−0.002	4.5%	−0.002	0.0034

The T-learner ranks customers best (Qini 0.245). That is 2.5 times the conversion model (0.096), which targets customers who are simply likely to buy.

Qini and AUUC normalized (random = 0, perfect = 1); overall observed uplift on the test set 0.0495; last two columns: mean and SD over 5 splits

QINI CURVES

Incremental conversions as more customers are contacted, in order of each model's score



What we see

The three boosted uplift models almost overlap and peak at about 8,000 extra conversions near 65% of customers.

After that the curves fall, because the last customers buy less when they get the offer.

The conversion model starts below zero: its top customers are often the ones the offer pushes away.

Held-out test set, 180,000 customers; curves sampled every 2% of customers; grey line = random targeting

STABILITY OVER FIVE SPLITS

Qini coefficient on the test part of five different stratified 70 / 30 splits

MODEL	SEED 42	SEED 7	SEED 123	SEED 2024	SEED 99	MEAN	SD
T-learner	**0.245**	**0.247**	**0.242**	**0.240**	**0.245**	**0.244**	**0.0026**
S-learner	0.243	0.245	0.241	0.233	0.242	0.241	0.0046
Class transformation	0.232	0.234	0.231	0.226	0.229	0.230	0.0027
Conversion model	0.097	0.092	0.095	0.098	0.098	0.096	0.0026
Linear T-learner	0.041	0.033	0.036	0.035	0.034	0.036	0.0030
Random	−0.003	−0.007	−0.000	−0.002	0.002	−0.002	0.0034

5 of 5 splits where the T-learner beats random

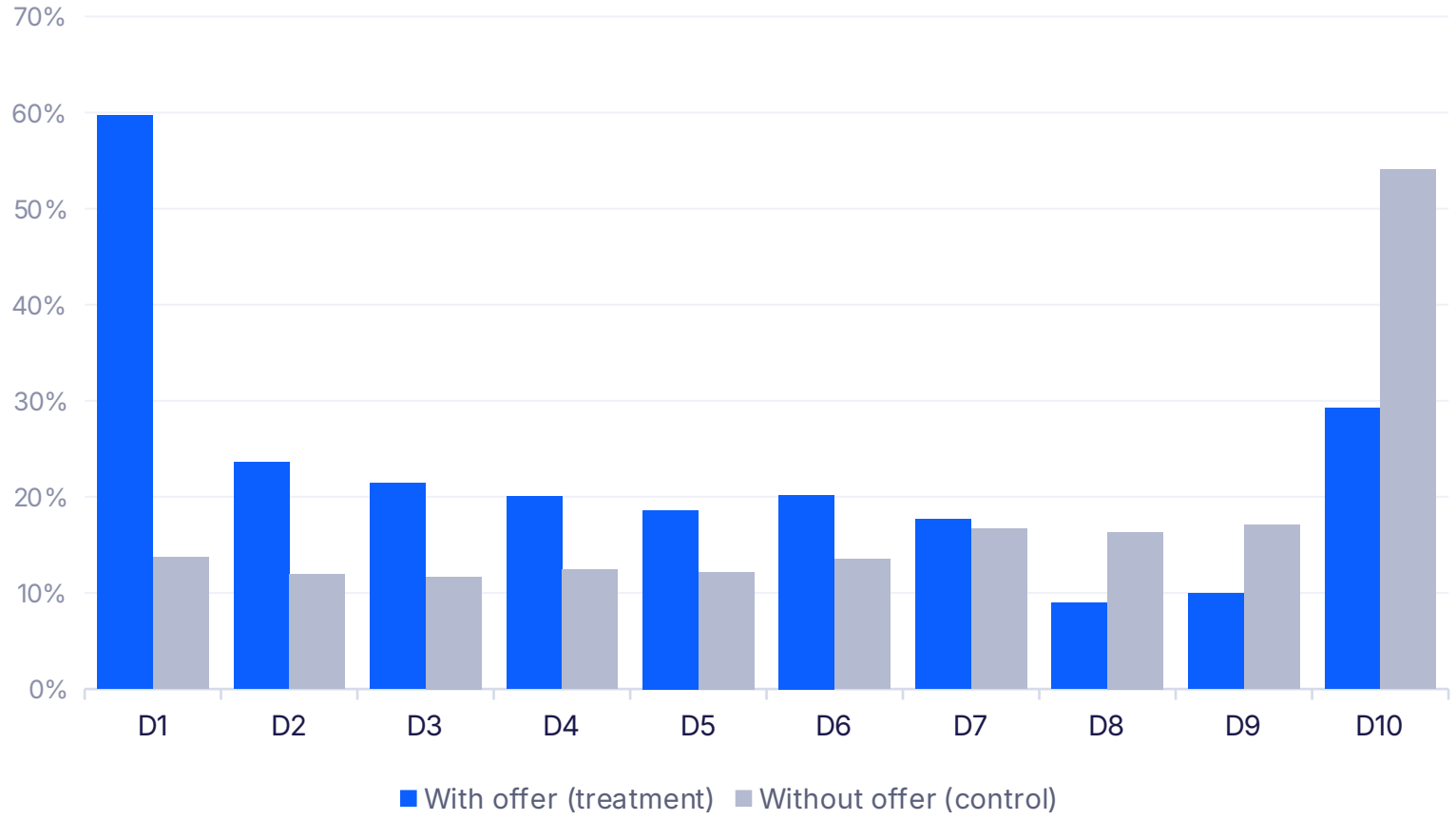
0.0026 SD of its Qini, the smallest of all models

T-learner selected as the final model

Each column is a different random seed for the split; every model re-trained with the tuned settings; SD = standard deviation over the 5 splits

OBSERVED UPLIFT BY DECILE

Test customers sorted by the T-learner's score and cut into 10 groups of 18,000 (D1 = highest)



	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10
UPLIFT	+0.46	+0.12	+0.10	+0.08	+0.06	+0.07	+0.01	−0.07	−0.07	−0.25

+46 pp

D1, persuadables

conversion 13.7% without and 59.7% with the offer

−25 pp

D10, sleeping dogs

conversion 54.1% without and only 29.3% with the offer

Observed uplift = treatment conversion rate − control conversion rate inside each decile; held-out test set

CAMPAIGN EFFICIENCY

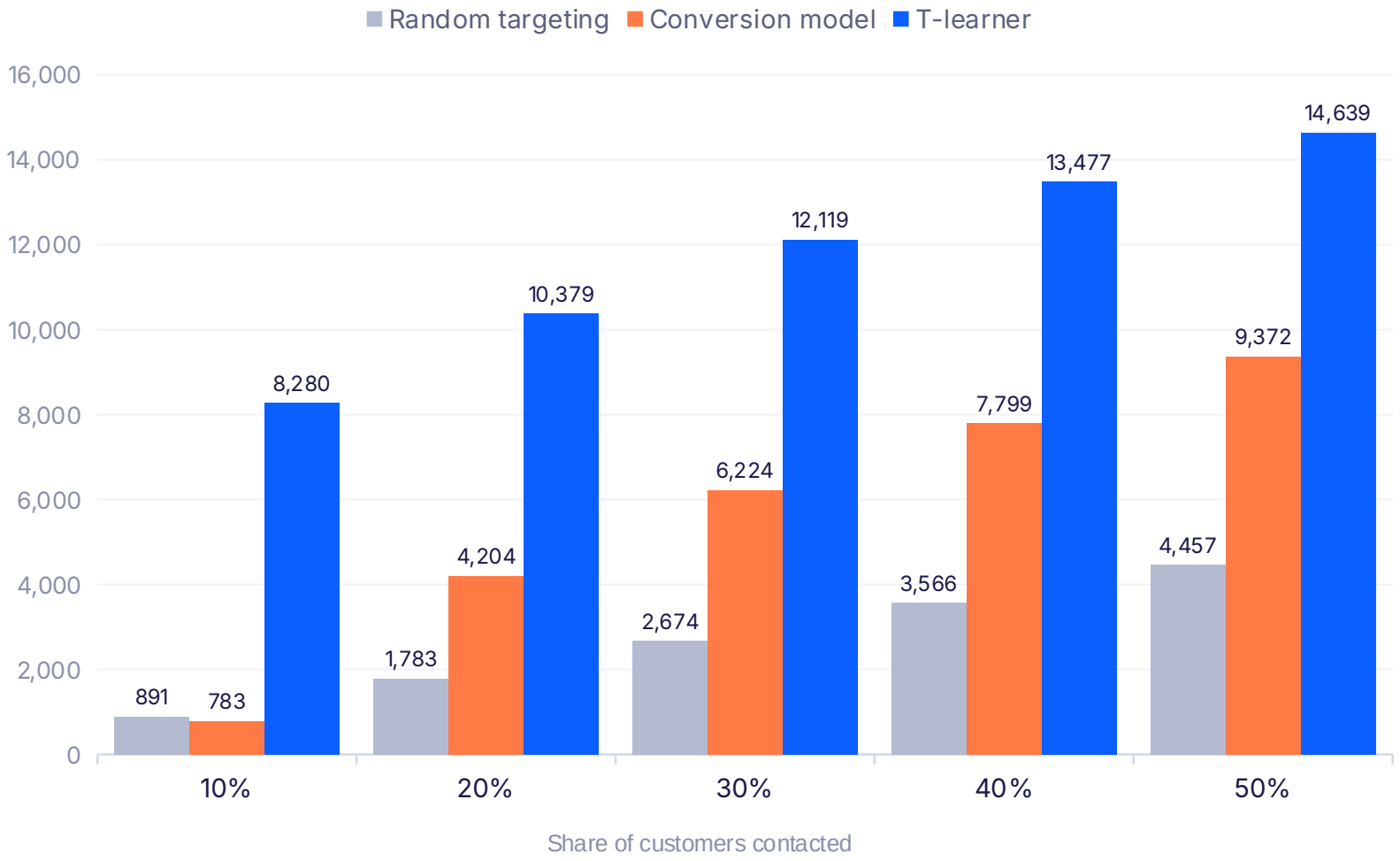
Estimated extra conversions when a share of the 180,000 test customers is contacted

AT A 30% BUDGET

4.5×

more extra conversions than random targeting

TARGETING	EXTRA CONV.
T-learner	**12,119**
Conversion model	6,224
Random	2,674



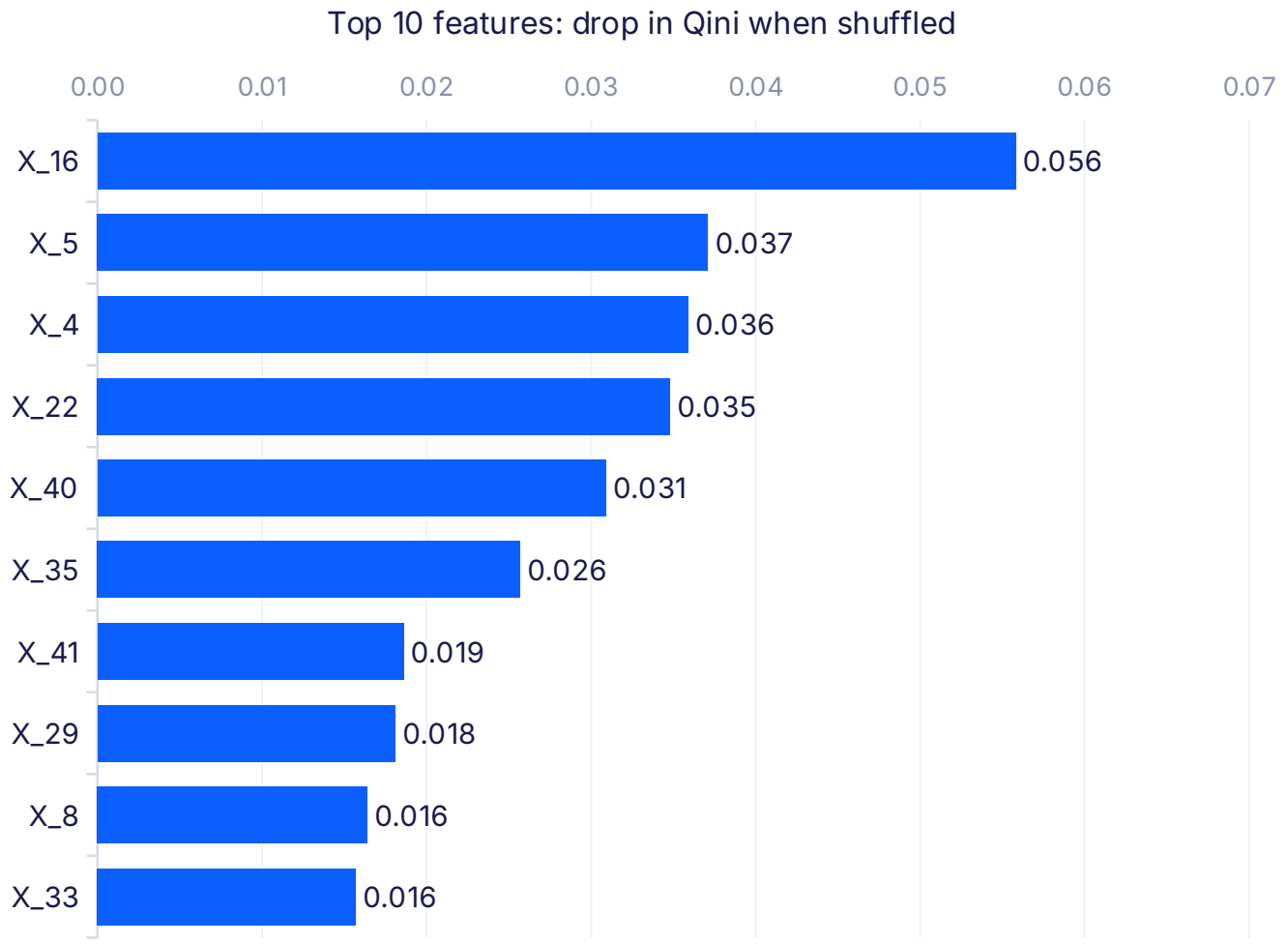
Extra conversions = customers contacted × observed uplift in the selected group; random targeting = customers contacted × 4.95%

NEGATIVE UPLIFT AND FEATURE INFLUENCE

35.2%
of test customers have negative predicted uplift

-11.5 pp
observed effect of the offer in this group

+13.8 pp
observed effect for all other customers



Permutation importance: drop in Qini when one feature is shuffled (60,000 test customers, 2 repeats); features are anonymized, so no business meaning is given

DISCUSSION

Why the T-learner won

- About 210,000 training rows per group is enough for two separate models.
- The S-learner has to learn the effect through one binary feature and can shrink it toward zero.
- The linear T-learner (Qini 0.041) shows the effect is non-linear.

Overfitting and underfitting

- Tuning used only training folds; the test set was untouched.
- min_samples_leaf 100 and L2 1.0 limit tree complexity.
- Qini SD below 0.005 on 5 splits for all boosted models. Only the linear model underfits.

Limitations and improvements

- Synthetic and anonymized data: effects may be stronger than in real campaigns, and features cannot be explained.
- One offer, binary outcome, same cost for every contact.
- Next: X-learner, causal forests, profit-based targeting.

Künzel et al. (2019) on S- and T-learners; results from the notebook

CONCLUSION

1 Target on uplift, not on the chance to buy

T-learner Qini 0.245 against 0.096 for the conversion model, positive on 5 of 5 splits.

2 Same budget, more effect

The top 30% by uplift show 22.4% observed uplift against 5.0% on average, about 4.5 times random targeting.

3 Know whom not to contact

35.2% of customers are expected to buy less if they get the offer.

Practical use

Any operator with a random control group can run this pipeline for upsell, cross-sell or retention offers and send them only to customers with positive uplift.

Future work

- X-learner, R-learner, causal forests
- Include contact cost and revenue
- Several offers at once
- A/B test of the list on a real campaign

All numbers from the held-out test set (180,000 customers)

REFERENCES

Ascarza, E. (2018). Retention futility: Targeting high-risk customers might be ineffective. Journal of Marketing Research, 55(1), 80–98.

Athey, S., & Imbens, G. (2016). Recursive partitioning for heterogeneous causal effects. PNAS, 113(27), 7353–7360.

Devriendt, F., Moldovan, D., & Verbeke, W. (2018). A literature survey and experimental evaluation of the state-of-the-art in uplift modeling. Big Data, 6(1), 13–41.

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. The Annals of Statistics, 29(5), 1189–1232.

Gutierrez, P., & Gérardy, J.-Y. (2017). Causal inference and uplift modelling: A review of the literature. PMLR, 67, 1–13.

Jaskowski, M., & Jaroszewicz, S. (2012). Uplift modeling for clinical trial data. ICML 2012 Workshop on Clinical Data Analysis.

Künzel, S. R., Sekhon, J. S., Bickel, P. J., & Yu, B. (2019). Metalearners for estimating heterogeneous treatment effects using machine learning. PNAS, 116(10), 4156–4165.

MegaFon. (2021). MegaFon Uplift Competition: Data [Data set]. ODS.ai. https://ods.ai/tracks/df21-megafon/competitions/megafon-df21-comp/data

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Radcliffe, N. J. (2007). Using control groups to target on predicted lift: Building and assessing uplift models. Direct Marketing Analytics Journal, 14–21.

Rubin, D. B. (1974). Estimating causal effects of treatments in randomized and nonrandomized studies. Journal of Educational Psychology, 66(5), 688–701.

Shevchenko, M. (n.d.). scikit-uplift (Version 0.5.1) [Computer software]. https://www.uplift-modeling.com

APA 7th edition; full list in the research paper

Uplift Modeling for Personalized Marketing in Telecommunications

Applied Machine Learning — Final Project
Students: Abilkhay Askar, Kolzhabay Olzhas
Group: Big Data Analysis 2406

README — how to run this project

Goal. Decide which telecom subscribers should receive a marketing offer so that *incremental* conversions are maximised. For every customer the notebook estimates the uplift `uplift(x) = P(Y=1 | X=x, T=1) − P(Y=1 | X=x, T=0)` and ranks customers by it.

Requirements. Python ≥ 3.9 with `numpy`, `pandas`, `scikit-learn`, `matplotlib` and `scikit-uplift` (`sklift`). Google Colab already has everything except `scikit-uplift`, which the first code cell installs automatically.

Data. MegaFon Uplift Competition dataset (600,000 labelled rows, 50 anonymised features). The loader cell reads `data/megafon_dataset.csv.gz` (run `python scripts/download_data.py` from the repository root) or `megafon_dataset.csv.gz` next to the notebook; otherwise it downloads the same file with `sklift.datasets.fetch_megafon()` (~270 MB, a few minutes on Colab).

Steps to reproduce.

1. Open the notebook in Google Colab (or Jupyter) and choose *Runtime → Run all*.
2. All random seeds are fixed (`SEED = 42`, repeated splits use seeds 42, 7, 123, 2024, 99). Results are identical on every run on the same library versions.
3. The full pipeline (loading → tuning → evaluation → targeting list) took 14 minutes on a 2-core machine (allow 20–30 minutes on free Colab); the last cell prints the measured runtime.

Outputs. Figures are written to the repository's `figures/` folder, tables to `results/`, and the final targeting list to `results/targeting_list_top30.csv.gz` (customer id, predicted uplift, rank, `target` flag). In Colab both folders are created next to the notebook.

Notebook structure. 1 Setup → 2 Load & validate → 3 Treatment/control → 4 Group balance → 5 Train/test split → 6 Hyper-parameter tuning by cross-validated Qini → 7 Models & evaluation → 8 Repeated splits & model selection → 9 Decile analysis → 10 Campaign efficiency → 11 Targeting list → 12 Negative-uplift profile → 13 Feature influence (permutation importance) → 14 Success-criteria check.

1. Setup

Install `scikit-uplift` if it is missing, import libraries, fix random seeds and define project-wide settings.

In [1]:
```
import sys, subprocess, importlib.util
if importlib.util.find_spec("sklift") is None:          # Colab: install scikit-uplift once
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", "scikit-uplift"])

import os, time, json, warnings
import numpy as np
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
import sklearn, sklift
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklift.models import SoloModel, TwoModels, ClassTransformation
from sklift.metrics import (qini_auc_score, uplift_auc_score, uplift_at_k,
                            qini_curve, uplift_by_percentile)

warnings.filterwarnings("ignore", category=FutureWarning)
T0 = time.time()
SEED = 42
SEEDS = [42, 7, 123, 2024, 99]      # repeated train/test splits
TEST_SIZE = 0.30
K = 0.30                            # campaign contact share
from pathlib import Path
# repo layout: notebooks/ sits next to data/, figures/ and results/; in Colab everything goes to the current folder
ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
FIG_DIR, RES_DIR = ROOT / "figures", ROOT / "results"
FIG_DIR.mkdir(exist_ok=True); RES_DIR.mkdir(exist_ok=True)

# consistent figure style
COLORS = {"S-learner": "#2a78d6", "T-learner": "#eb6834", "Class transformation": "#1baf7a",
          "T-learner (LogReg)": "#eda100", "Conversion model": "#e87ba4", "Random": "#8a8984"}
plt.rcParams.update({"figure.dpi": 110, "savefig.dpi": 200, "axes.spines.top": False,
                     "axes.spines.right": False, "axes.grid": True, "grid.alpha": 0.25,
                     "font.size": 10, "axes.titleweight": "bold"})
```

```
print(f"python {sys.version.split()[0]} | numpy {np.__version__} | pandas {pd.__version__} | "
      f"scikit-learn {sklearn.__version__} | scikit-uplift {sklift.__version__} | matplotlib {matplotlib.__version__}")
```

Out[1]:
```
python 3.13.18 | numpy 2.5.3 | pandas 3.0.5 | scikit-learn 1.9.1 | scikit-uplift 0.5.1 | matplotlib 3.11.2
```

2. Load and validate the data (Requirement 3.1-1)

The file contains the treatment flag, 50 anonymised numeric features `X_1` … `X_50` and the binary `conversion` outcome. The distributed file has no explicit subscriber id column, so the row index is used as `customer_id`.

In [2]:
```
CANDIDATES = [ROOT / "data" / "megafon_dataset.csv.gz", Path("megafon_dataset.csv.gz")]
DATA_PATH = next((p for p in CANDIDATES if p.exists()), None)
if DATA_PATH is not None:
    df = pd.read_csv(DATA_PATH)
else:
    from sklift.datasets import fetch_megafon
    b = fetch_megafon()
    df = pd.concat([b.data, b.treatment, b.target], axis=1)
df.insert(0, "customer_id", np.arange(len(df)))

FEATURES = [f"X_{i}" for i in range(1, 51)]
print("Shape (rows, columns):", df.shape)
print("Feature columns:", len(FEATURES))
print("Missing values (total):", int(df.isna().sum().sum()))
print("Duplicate rows (excluding id):", int(df.drop(columns="customer_id").duplicated().sum()))
print("Column types:", df.dtypes.astype(str).value_counts().to_dict())
display(df[["customer_id", "treatment_group"] + FEATURES[:6] + ["X_50", "conversion"]].head().round(3))
display(df[FEATURES[:8]].describe().T.round(3))
```

Out[2]:
```
Shape (rows, columns): (600000, 53)
Feature columns: 50
Missing values (total): 0
Duplicate rows (excluding id): 0
Column types: {'float64': 50, 'int64': 2, 'str': 1}
```

Out[2]:

	customer_id	treatment_group	X_1	X_2	X_3	X_4	X_5	X_6	X_50	conversion
0	0	control	39.397	-0.187	19.525	21.250	55.291	182.967	16.231	0
1	1	control	38.988	0.820	-42.065	-48.271	-33.171	179.459	10.732	0
2	2	treatment	-16.693	1.845	-8.615	-18.819	-22.271	-116.290	14.368	0
3	3	treatment	-72.040	-0.227	39.803	16.441	-1.113	68.128	11.634	0
4	4	treatment	18.297	0.996	24.465	-34.152	24.623	-155.456	-11.884	0

Out[2]:

	count	mean	std	min	25%	50%	75%	max
X_1	600000.0	-3.759	54.882	-271.659	-40.693	-3.955	33.175	250.812
X_2	600000.0	0.000	0.999	-4.372	-0.673	0.001	0.673	5.062
X_3	600000.0	0.356	31.804	-148.871	-20.758	0.373	21.496	170.053
X_4	600000.0	-1.004	45.291	-244.447	-30.645	-0.585	29.028	235.096
X_5	600000.0	3.377	53.398	-302.574	-31.865	3.721	38.989	284.916
X_6	600000.0	-6.396	140.874	-683.126	-100.762	-6.357	88.160	656.482
X_7	600000.0	-2.254	59.810	-322.732	-42.314	-2.264	37.710	293.910
X_8	600000.0	-6.433	74.841	-506.203	-54.841	-6.416	41.963	550.526

3. Treatment and control groups (Requirement 3.1-2)

`T = 1` for customers who received the offer, `T = 0` for the control group. The difference in conversion rates is the average treatment effect (ATE) of the campaign.

In [3]:
```
df["T"] = (df["treatment_group"] == "treatment").astype(int)
X, y, t = df[FEATURES], df["conversion"].astype(int), df["T"]
assert set(df["treatment_group"].unique()) == {"treatment", "control"}

grp = df.groupby("treatment_group")["conversion"].agg(n="size", conversions="sum", conversion_rate="mean")
grp["share"] = grp["n"] / len(df)
display(grp.round(4))

p1, p0 = y[t == 1].mean(), y[t == 0].mean()
n1, n0 = (t == 1).sum(), (t == 0).sum()
ate = p1 - p0
se = np.sqrt(p1 * (1 - p1) / n1 + p0 * (1 - p0) / n0)
print(f"Average treatment effect (ATE) = {ate:.4f}  (95% CI {ate - 1.96*se:.4f} to {ate + 1.96*se:.4f})")
print(f"Relative lift = {ate / p0:.1%}")

fig, ax = plt.subplots(figsize=(5.2, 3.2))
bars = ax.bar(["Control (T=0)", "Treatment (T=1)"], [p0, p1], color=["#8a8984", "#2a78d6"], width=0.55)
for b_, v in zip(bars, [p0, p1]):
    ax.text(b_.get_x() + b_.get_width() / 2, v + 0.004, f"{v:.1%}", ha="center")
ax.set_ylabel("Conversion rate"); ax.set_ylim(0, 0.28)
ax.set_title("Conversion rate by group")
fig.tight_layout(); fig.savefig(FIG_DIR / "fig1_group_conversion.png"); plt.show()
```

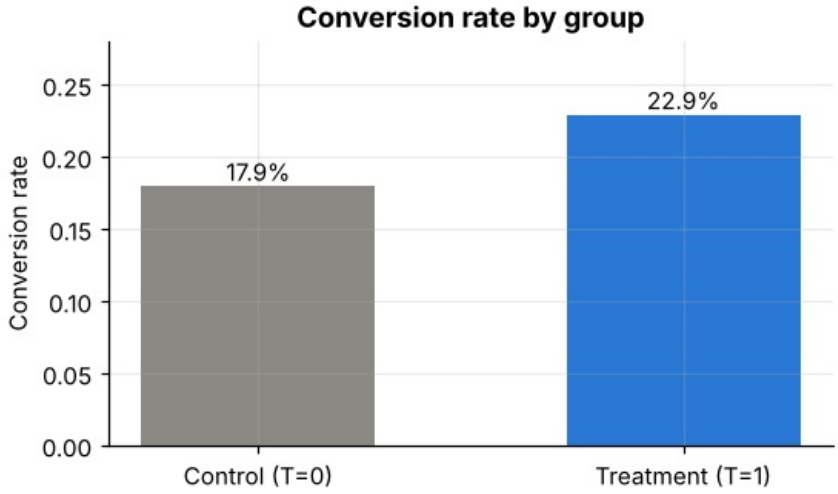
Out[3]:

treatment_group	n	conversions	conversion_rate	share
control	299632	53754	0.1794	0.4994
treatment	300368	68760	0.2289	0.5006

Out[3]:
```
Average treatment effect (ATE) = 0.0495  (95% CI 0.0475 to 0.0516)
```

```
Relative lift = 27.6%
```

Out[3]:



4. Group balance check (Requirement 3.1-3)

If the offer was assigned at random, every feature should have almost the same distribution in both groups. We use the standardised mean difference `SMD = (mean_T − mean_C) / sqrt((var_T + var_C) / 2)` with the usual threshold |SMD| < 0.1.

In [4]:

```
mt, mc = X[t == 1].mean(), X[t == 0].mean()
vt, vc = X[t == 1].var(), X[t == 0].var()
smd = ((mt - mc) / np.sqrt((vt + vc) / 2)).rename("SMD")
print(f"Max |SMD| = {smd.abs().max():.4f} ({smd.abs().idxmax()})")
print("Features with |SMD| >= 0.1:", int((smd.abs() >= 0.1).sum()))
display(smd.abs().sort_values(ascending=False).head(5).to_frame("|SMD|").round(4))

fig, ax = plt.subplots(figsize=(9, 3.2))
ax.bar(range(1, 51), smd.abs().values, color="#2a78d6", width=0.7)
ax.axhline(0.1, color="#d03b3b", ls="--", lw=1.2, label="threshold 0.1")
ax.set_xlabel("Feature index (X_1 … X_50)"); ax.set_ylabel("|SMD|")
ax.set_title("Treatment vs control balance for all 50 features"); ax.set_ylim(0, 0.11); ax.legend()
fig.tight_layout(); fig.savefig(FIG_DIR / "fig2_balance_smd.png"); plt.show()
```

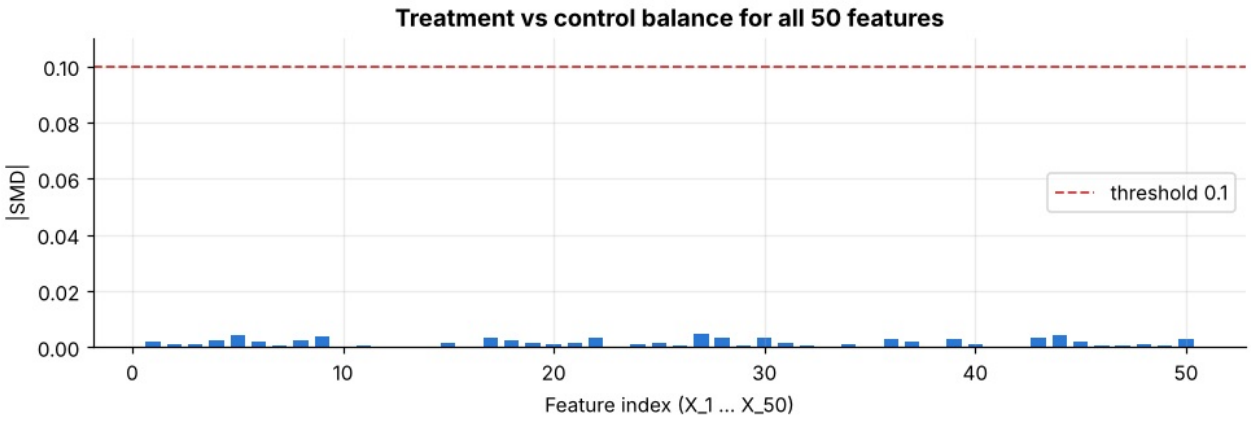
Out[4]:

```
Max |SMD| = 0.0046 (X_27)
Features with |SMD| >= 0.1: 0
```

Out[4]:

	|SMD|
X_27	0.0046
X_5	0.0045
X_44	0.0044
X_9	0.0036
X_28	0.0036

Out[4]:



5. Train/test split (Requirement 3.1-4)

30 % of the rows are held out. The split is stratified on the four combinations of treatment and conversion, so both proportions are preserved. The test set is never used for training or tuning.

In [5]:

```
def split(seed):
    strata = t * 2 + y
    return train_test_split(np.arange(len(df)), test_size=TEST_SIZE, stratify=strata, random_state=seed)
```

```
idx_tr, idx_te = split(SEED)
X_tr, X_te = X.iloc[idx_tr], X.iloc[idx_te]
y_tr, y_te = y.iloc[idx_tr], y.iloc[idx_te]
t_tr, t_te = t.iloc[idx_tr], t.iloc[idx_te]

split_tab = pd.DataFrame({
    "rows": [len(df), len(idx_tr), len(idx_te)],
    "treatment_share": [t.mean(), t_tr.mean(), t_te.mean()],
    "conversion_rate": [y.mean(), y_tr.mean(), y_te.mean()],
    "observed_uplift": [ate, y_tr[t_tr == 1].mean() - y_tr[t_tr == 0].mean(),
                        y_te[t_te == 1].mean() - y_te[t_te == 0].mean()]},
    index=["full", "train", "test"])
display(split_tab.round(4))
print("Max deviation of treatment share from full data (pp):",
      round(100 * max(abs(t_tr.mean() - t.mean()), abs(t_te.mean() - t.mean())), 4))
```

Out[5]:

	rows	treatment_share	conversion_rate	observed_uplift
full	600000	0.5006	0.2042	0.0495
train	420000	0.5006	0.2042	0.0495
test	180000	0.5006	0.2042	0.0495

Out[5]:
```
Max deviation of treatment share from full data (pp): 0.0002
```

6. Model definitions and hyper-parameter tuning by cross-validated Qini (Nice-to-have 3.2-2)

Approaches

- **S-learner** (`SoloModel`): one classifier on `[X, T]`; uplift = f(x, 1) − f(x, 0).
- **T-learner** (`TwoModels`, vanilla): one classifier on the treated rows and one on the control rows; uplift = $f_1(x) - f_0(x)$.
- **Class transformation** (`ClassTransformation`): new label Z = 1 if (T=1, Y=1) or (T=0, Y=0); with a 50/50 treatment split, uplift = 2·P(Z=1|x) − 1.
- **T-learner (LogReg)**: the same T-learner with standardised logistic regression, a linear reference.
- **Baselines**: random scores, and a conventional *conversion model* (the same gradient-boosting classifier trained on X only, ranking customers by P(Y=1|x)).

Base learner. `HistGradientBoostingClassifier` (histogram gradient-boosted trees). Settings are chosen by the mean 3-fold cross-validated Qini coefficient **on the training set only** — not by accuracy.

In [6]:
```
def hgb(lr=0.1, leaves=31, iters=200):
    return HistGradientBoostingClassifier(learning_rate=lr, max_leaf_nodes=leaves, max_iter=iters,
                                          min_samples_leaf=100, l2_regularization=1.0,
                                          early_stopping=False, random_state=SEED)

def make_model(name, params):
    if name == "S-learner":
        return SoloModel(estimator=hgb(**params), method="dummy")
    if name == "T-learner":
        return TwoModels(estimator_trmnt=hgb(**params), estimator_ctrl=hgb(**params), method="vanilla")
    if name == "Class transformation":
        return ClassTransformation(estimator=hgb(**params))
    if name == "T-learner (LogReg)":
        lr_ = lambda: make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
        return TwoModels(estimator_trmnt=lr_(), estimator_ctrl=lr_(), method="vanilla")
    raise ValueError(name)

FITTED = {}   # fitted models from the main split (filled in Section 7)

def fit_score(name, params, Xa, ya, ta, Xb, keep=False):
    """Fit an uplift model (or a baseline) on (Xa, ya, ta) and return uplift-ranking scores for Xb."""
    if name == "Random":
        return np.random.RandomState(SEED).rand(len(Xb))
    if name == "Conversion model":
        return hgb(**params).fit(Xa, ya).predict_proba(Xb)[:, 1]
    model = make_model(name, params).fit(Xa, ya, ta)
    scores = model.predict(Xb)
    if keep:
        FITTED[name] = model
    return scores

GRID = [dict(lr=0.1, leaves=31, iters=200), dict(lr=0.05, leaves=31, iters=300),
        dict(lr=0.1, leaves=15, iters=200), dict(lr=0.1, leaves=63, iters=150)]
TUNED = ["S-learner", "T-learner", "Class transformation"]

t_start = time.time()
skf = StratifiedKFold(n_splits=3, shuffle=True, random_state=SEED)
cv_rows = []
for name in TUNED:
    for gi, params in enumerate(GRID):
        scores = []
        for f_tr, f_va in skf.split(X_tr, t_tr * 2 + y_tr):
            u = fit_score(name, params, X_tr.iloc[f_tr], y_tr.iloc[f_tr], t_tr.iloc[f_tr], X_tr.iloc[f_va])
            scores.append(qini_auc_score(y_tr.iloc[f_va], u, t_tr.iloc[f_va]))
        cv_rows.append({"model": name, "config": gi, **params,
                        "cv_qini_mean": np.mean(scores), "cv_qini_std": np.std(scores)})
cv = pd.DataFrame(cv_rows)
display(cv.round(4))
BEST = {m: GRID[int(cv[cv.model == m].sort_values("cv_qini_mean").iloc[-1]["config"])] for m in TUNED}
BEST["Conversion model"] = BEST["S-learner"]
for m in TUNED:
```

```
    print(f"{m:22s} selected config: {BEST[m]}")
print(f"Tuning time: {(time.time() - t_start) / 60:.1f} min")
cv.to_csv(RES_DIR / "cv_tuning.csv", index=False)
```

Out[6]:

	model	config	lr	leaves	iters	cv_qini_mean	cv_qini_std
0	S-learner	0	0.10	31	200	0.2313	0.0012
1	S-learner	1	0.05	31	300	0.2310	0.0015
2	S-learner	2	0.10	15	200	0.2179	0.0020
3	S-learner	3	0.10	63	150	0.2365	0.0007
4	T-learner	0	0.10	31	200	0.2364	0.0015
5	T-learner	1	0.05	31	300	0.2361	0.0024
6	T-learner	2	0.10	15	200	0.2292	0.0015
7	T-learner	3	0.10	63	150	0.2380	0.0003
8	Class transformation	0	0.10	31	200	0.2184	0.0025
9	Class transformation	1	0.05	31	300	0.2233	0.0033
10	Class transformation	2	0.10	15	200	0.2177	0.0021
11	Class transformation	3	0.10	63	150	0.2179	0.0022

Out[6]:

```
S-learner             selected config: {'lr': 0.1, 'leaves': 63, 'iters': 150}
T-learner             selected config: {'lr': 0.1, 'leaves': 63, 'iters': 150}
Class transformation  selected config: {'lr': 0.05, 'leaves': 31, 'iters': 300}
Tuning time: 9.6 min
```

7. Estimate uplift and evaluate on the held-out test set (Requirements 3.1-5, 3.1-6; Nice-to-have 3.2-1, 3.2-3)

Metrics (all from `sklift.metrics`, computed on the 180,000 test customers):

- **Qini coefficient** – area between the model's Qini curve and the random line, normalised by the perfect model (random = 0, perfect = 1).
- **AUUC** – the same normalisation applied to the uplift curve.
- **Uplift@30 %** – treatment conversion rate minus control conversion rate among the 30 % of customers with the highest scores.

In [7]:

```
MODELS = ["S-learner", "T-learner", "Class transformation", "T-learner (LogReg)", "Conversion model", "Random"]
scores_te, fit_time = {}, {}
for name in MODELS:
    s0 = time.time()
    scores_te[name] = fit_score(name, BEST.get(name, GRID[0]), X_tr, y_tr, t_tr, X_te, keep=True)
    fit_time[name] = time.time() - s0

overall_uplift_te = y_te[t_te == 1].mean() - y_te[t_te == 0].mean()
res = pd.DataFrame({
    "Qini coefficient": {m: qini_auc_score(y_te, s, t_te) for m, s in scores_te.items()},
    "AUUC": {m: uplift_auc_score(y_te, s, t_te) for m, s in scores_te.items()},
    "Uplift@30%": {m: uplift_at_k(y_te, s, t_te, strategy="overall", k=K) for m, s in scores_te.items()},
    "Fit+predict time (s)": fit_time}).loc[MODELS]
print(f"Observed uplift on the whole test set: {overall_uplift_te:.4f}")
display(res.round(4))
res.to_csv(RES_DIR / "test_metrics.csv")
```

Out[7]:

```
Observed uplift on the whole test set: 0.0495
```

Out[7]:

	Qini coefficient	AUUC	Uplift@30%	Fit+predict time (s)
S-learner	0.2430	0.1679	0.2193	11.9013
T-learner	0.2450	0.1695	0.2244	13.5178
Class transformation	0.2315	0.1599	0.2165	14.6379
T-learner (LogReg)	0.0405	0.0282	0.0852	2.0673
Conversion model	0.0965	0.0666	0.1153	10.7173
Random	-0.0033	-0.0023	0.0454	0.0011

Qini curves. Each curve shows the cumulative number of incremental conversions when customers are contacted in the order given by the model. The dashed diagonal is random targeting.

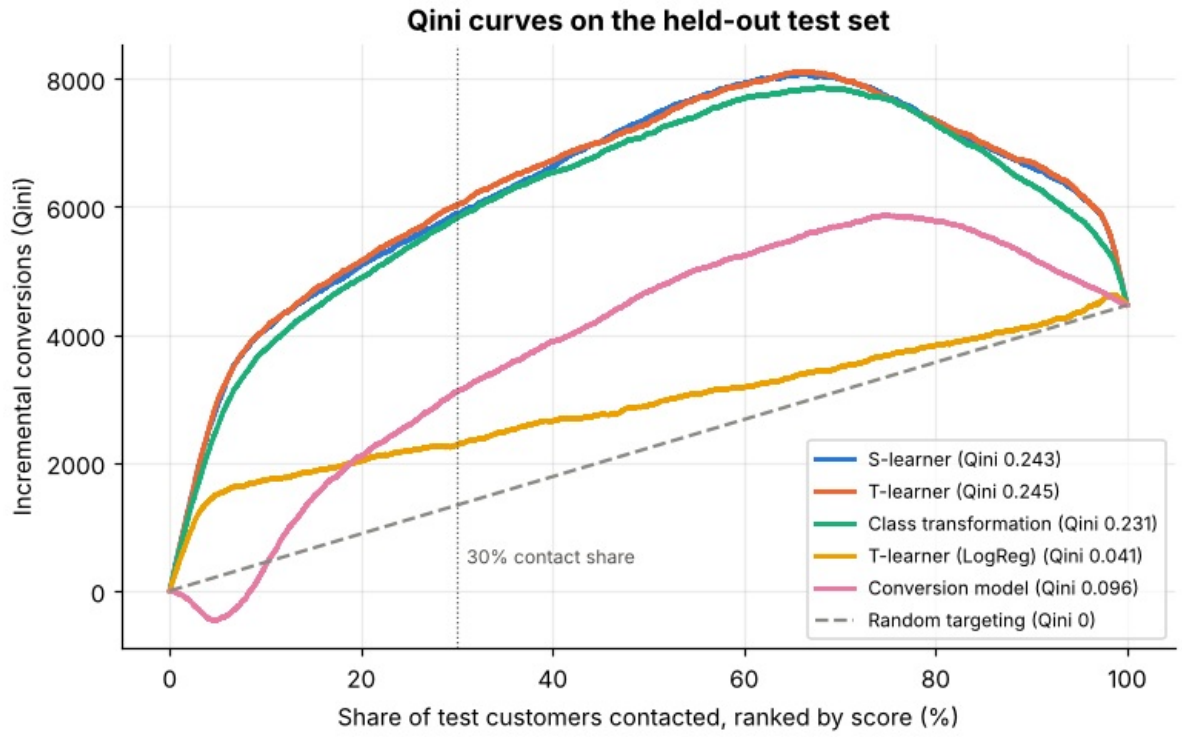
In [8]:

```
fig, ax = plt.subplots(figsize=(7.2, 4.6))
for name in MODELS:
    if name == "Random":
        continue
    xs, ys = qini_curve(y_te, scores_te[name], t_te)
    ax.plot(xs / len(y_te) * 100, ys, lw=2, color=COLORS[name],
            label=f"{name} (Qini {res.loc[name, 'Qini coefficient']:.3f})")
ax.plot([0, 100], [0, ys[-1]], ls="--", color=COLORS["Random"], lw=1.5, label="Random targeting (Qini 0)")
ax.axvline(30, color="#52514e", lw=0.8, ls=":")
ax.text(31, ax.get_ylim()[1] * 0.05, "30% contact share", color="#52514e", fontsize=8)
ax.set_xlabel("Share of test customers contacted, ranked by score (%)")
ax.set_ylabel("Incremental conversions (Qini)")
ax.set_title("Qini curves on the held-out test set")
ax.legend(fontsize=8, loc="lower right")
fig.tight_layout(); fig.savefig(FIG_DIR / "fig3_qini_curves.png"); plt.show()

# save the curves at 2% steps for the presentation
grid_pct = np.arange(0, 101, 2)
curve_tab = {"share_pct": grid_pct}
for name in MODELS[:-1]:
```

```
    xs, ys = qini_curve(y_te, scores_te[name], t_te)
    curve_tab[name] = np.interp(grid_pct, xs / len(y_te) * 100, ys)
curve_tab["Random"] = grid_pct / 100 * ys[-1]
pd.DataFrame(curve_tab).to_csv(RES_DIR / "qini_curves.csv", index=False)
```

Out[8]:



8. Stability over five repeated train/test splits and final model selection (Success criterion 7)

Every approach is re-trained on five different stratified 70/30 splits (with the tuned settings). The final model is the approach with the **highest mean Qini coefficient**.

In [9]:

```
rep_rows = []
t_start = time.time()
for sd in SEEDS:
    a, b_ = split(sd)
    for name in MODELS:
        u = fit_score(name, BEST.get(name, GRID[0]), X.iloc[a], y.iloc[a], t.iloc[a], X.iloc[b_])
        rep_rows.append({"seed": sd, "model": name,
                         "qini": qini_auc_score(y.iloc[b_], u, t.iloc[b_]),
                         "uplift_at_30": uplift_at_k(y.iloc[b_], u, t.iloc[b_], strategy="overall", k=K)})
rep = pd.DataFrame(rep_rows)
summary = rep.groupby("model").agg(qini_mean=("qini", "mean"), qini_std=("qini", "std"),
                                   qini_min=("qini", "min"), positive_splits=("qini", lambda s: int((s > 0).sum())),
                                   uplift30_mean=("uplift_at_30", "mean")).loc[MODELS]
display(summary.round(4))
rep.to_csv(RES_DIR / "repeated_splits.csv", index=False)
print(f"Repeated-split time: {(time.time() - t_start) / 60:.1f} min")

FINAL = summary.drop(["Random", "Conversion model"])["qini_mean"].idxmax()
print("Selected final model:", FINAL)

fig, ax = plt.subplots(figsize=(7.2, 3.6))
order = summary.sort_values("qini_mean").index
for i, name in enumerate(order):
    vals = rep[rep.model == name]["qini"].values
    ax.scatter(vals, [i] * len(vals), color=COLORS[name], s=36, alpha=0.8, zorder=3)
    ax.plot([vals.mean()] * 2, [i - 0.3, i + 0.3], color="#0b0b0b", lw=2, zorder=4)
ax.axvline(0, color="#8a8984", ls="--", lw=1)
ax.set_yticks(range(len(order))); ax.set_yticklabels(order)
ax.set_xlabel("Qini coefficient on the test split (dots = 5 seeds, bar = mean)")
ax.set_title("Stability of the Qini coefficient across repeated splits")
fig.tight_layout(); fig.savefig(FIG_DIR / "fig4_repeated_splits.png"); plt.show()
```

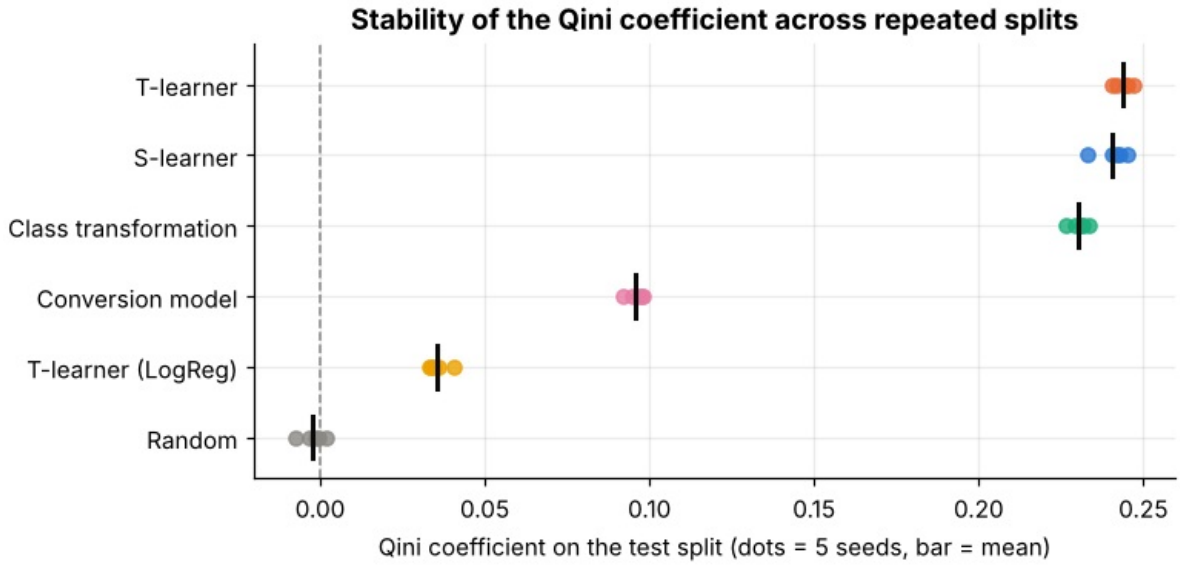
Out[9]:

model	qini_mean	qini_std	qini_min	positive_splits	uplift30_mean
S-learner	0.2407	0.0046	0.2330	5	0.2186
T-learner	0.2438	0.0026	0.2405	5	0.2227
Class transformation	0.2304	0.0027	0.2264	5	0.2145
T-learner (LogReg)	0.0356	0.0030	0.0332	5	0.0811
Conversion model	0.0958	0.0026	0.0919	5	0.1153
Random	-0.0021	0.0034	-0.0073	1	0.0479

Out[9]:

```
Repeated-split time: 4.5 min
Selected final model: T-learner
```

Out[9]:



9. Observed conversion rates by decile of predicted uplift (Nice-to-have 3.2-3)

Test customers are sorted by the final model's score and cut into ten equal groups. In each decile we report the *observed* treatment and control conversion rates; their difference is the observed uplift. A good model shows high observed uplift in the first deciles and low or negative uplift in the last ones.

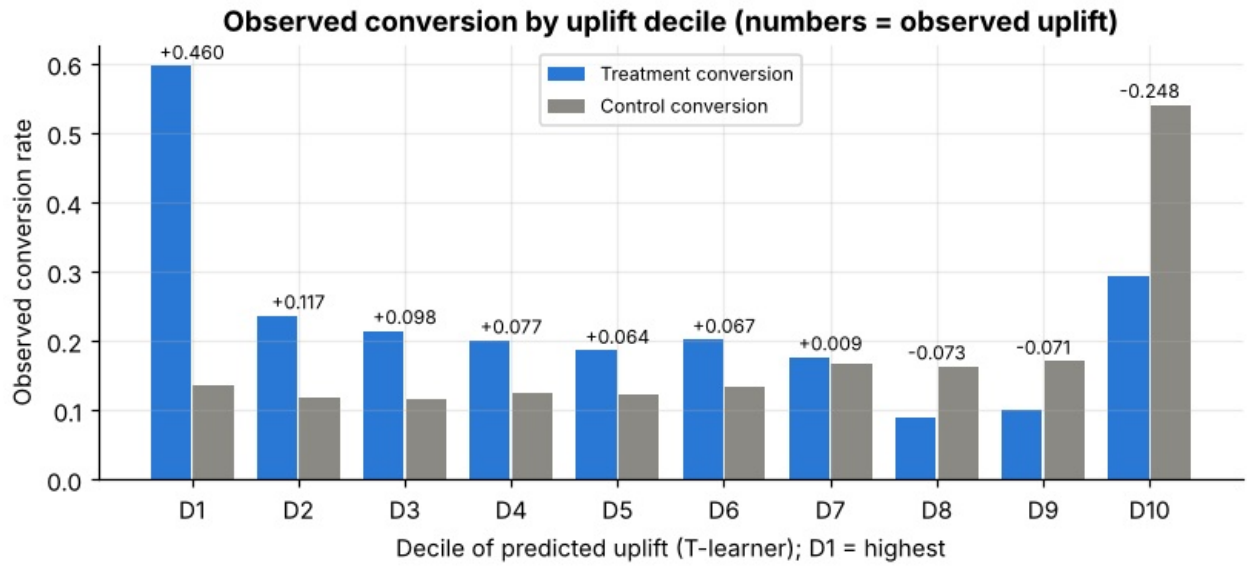
In [10]:
```
u_final = scores_te[FINAL]
dec = uplift_by_percentile(y_te, u_final, t_te, strategy="overall", bins=10, total=False)
dec.index = [f"D{i}" for i in range(1, 11)]
display(dec.round(4))
dec.to_csv(RES_DIR / "deciles_final_model.csv")

fig, ax = plt.subplots(figsize=(7.6, 3.6))
xi = np.arange(10)
ax.bar(xi - 0.2, dec["response_rate_treatment"], 0.38, color="#2a78d6", label="Treatment conversion")
ax.bar(xi + 0.2, dec["response_rate_control"], 0.38, color="#8a8984", label="Control conversion")
ax2_vals = dec["uplift"].values
for i, v in enumerate(ax2_vals):
    ax.text(i, max(dec["response_rate_treatment"].iloc[i], dec["response_rate_control"].iloc[i]) + 0.01,
            f"{v:+.3f}", ha="center", fontsize=8)
ax.set_xticks(xi); ax.set_xticklabels(dec.index)
ax.set_xlabel(f"Decile of predicted uplift ({FINAL}); D1 = highest")
ax.set_ylabel("Observed conversion rate"); ax.legend(fontsize=8)
ax.set_title("Observed conversion by uplift decile (numbers = observed uplift)")
fig.tight_layout(); fig.savefig(FIG_DIR / "fig5_deciles.png"); plt.show()
```

Out[10]:

	n_treatment	n_control	response_rate_treatment	response_rate_control	uplift
D1	8917	9083	0.5967	0.1367	0.4600
D2	8946	9054	0.2362	0.1190	0.1172
D3	8994	9006	0.2145	0.1168	0.0977
D4	9059	8941	0.2006	0.1238	0.0768
D5	8961	9039	0.1861	0.1218	0.0643
D6	9130	8870	0.2018	0.1347	0.0670
D7	9027	8973	0.1764	0.1669	0.0094
D8	9038	8962	0.0897	0.1631	-0.0734
D9	9019	8981	0.0993	0.1708	-0.0715
D10	9019	8981	0.2926	0.5409	-0.2483

Out[10]:



10. Campaign efficiency for different contact shares (Nice-to-have 3.2-4)

Incremental conversions in a contacted group of size n are estimated as n × (treatment rate − control rate) observed in that group. Random targeting of the same n customers is expected to give n × (overall test uplift).

In [11]:

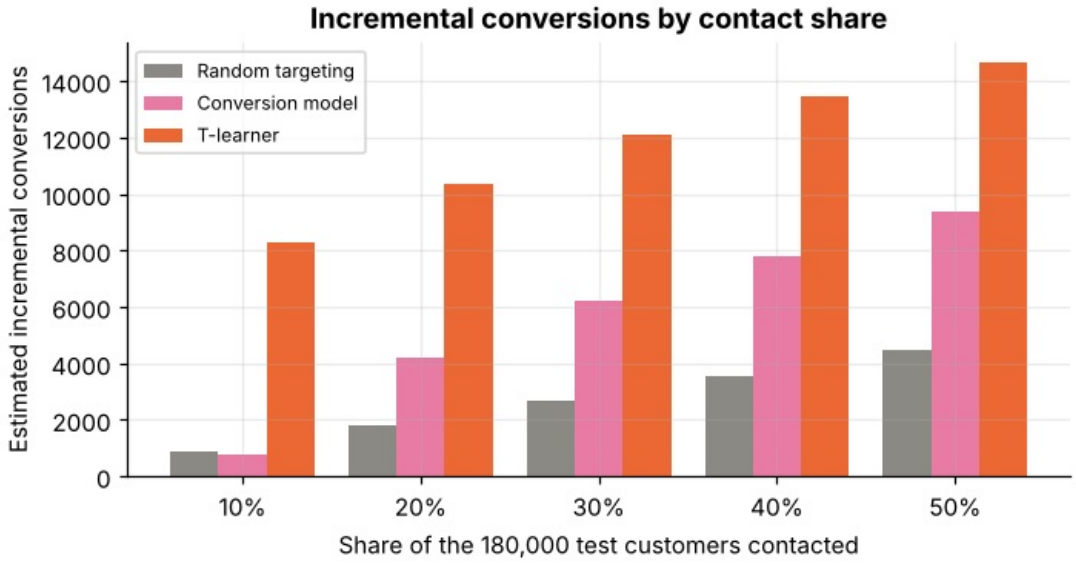
```
eff_rows = []
for k in [0.1, 0.2, 0.3, 0.4, 0.5]:
    n_sel = int(round(k * len(y_te)))
    row = {"contact_share": f"{int(k*100)}%", "customers_contacted": n_sel,
           "random_targeting": n_sel * overall_uplift_te}
    for name in [FINAL, "Conversion model"]:
        row[name] = n_sel * uplift_at_k(y_te, scores_te[name], t_te, strategy="overall", k=k)
    eff_rows.append(row)
eff = pd.DataFrame(eff_rows).set_index("contact_share")
eff[f"gain_vs_random ({FINAL})"] = eff[FINAL] / eff["random_targeting"]
display(eff.round(1))
eff.to_csv(RES_DIR / "campaign_efficiency.csv")

fig, ax = plt.subplots(figsize=(6.6, 3.6))
xi = np.arange(len(eff)); w = 0.27
ax.bar(xi - w, eff["random_targeting"], w, color=COLORS["Random"], label="Random targeting")
ax.bar(xi, eff["Conversion model"], w, color=COLORS["Conversion model"], label="Conversion model")
ax.bar(xi + w, eff[FINAL], w, color=COLORS[FINAL], label=FINAL)
ax.set_xticks(xi); ax.set_xticklabels(eff.index)
ax.set_xlabel("Share of the 180,000 test customers contacted")
ax.set_ylabel("Estimated incremental conversions"); ax.legend(fontsize=8)
ax.set_title("Incremental conversions by contact share")
fig.tight_layout(); fig.savefig(FIG_DIR / "fig6_campaign_efficiency.png"); plt.show()
```

Out[11]:

contact_share	customers_contacted	random_targeting	T-learner	Conversion model	gain_vs_random (T-learner)
10%	18000	891.4	8279.8	782.7	9.3
20%	36000	1782.8	10379.1	4204.0	5.8
30%	54000	2674.3	12119.2	6224.0	4.5
40%	72000	3565.7	13477.1	7799.3	3.8
50%	90000	4457.1	14638.8	9371.9	3.3

Out[11]:



11. Targeting list (Requirement 3.1-7)

All test customers are ranked by the final model's predicted uplift; the top 30 % are marked `target`. For the S- and T-learner the two potential-outcome probabilities are also reported, which reproduces the input/output examples from the proposal.

In [12]:
```
out = pd.DataFrame({"customer_id": df["customer_id"].iloc[idx_te].values, "predicted_uplift": u_final})
if FINAL in ("S-learner", "T-learner"):
    mdl = FITTED[FINAL]
    mdl.predict(X_te)                       # refreshes the stored per-arm probabilities for the test set
    out["p_with_offer"] = mdl.trmnt_preds_
    out["p_without_offer"] = mdl.ctrl_preds_
out = out.sort_values("predicted_uplift", ascending=False).reset_index(drop=True)
out["rank"] = np.arange(1, len(out) + 1)
n_target = int(round(K * len(out)))
out["decision"] = np.where(out["rank"] <= n_target, "target", "do not target")
out.to_csv(RES_DIR / "targeting_list_top30.csv.gz", index=False)
print(f"Customers ranked: {len(out):,}; marked 'target': {n_target:,} ({K:.0%})")
print(f"Missing or infinite scores: {int((~np.isfinite(out['predicted_uplift'])).sum())}")
display(out.head(5).round(4))
display(out.iloc[[n_target - 1, n_target, len(out) - 1]].round(4))
```

Out[12]:
```
Customers ranked: 180,000; marked 'target': 54,000 (30%)
Missing or infinite scores: 0
```

Out[12]:

	customer_id	predicted_uplift	p_with_offer	p_without_offer	rank	decision
0	435517	0.8340	0.9004	0.0664	1	target
1	512292	0.8324	0.9124	0.0799	2	target
2	361952	0.8315	0.8917	0.0602	3	target
3	536669	0.8249	0.9009	0.0760	4	target
4	246842	0.8246	0.9290	0.1044	5	target

Out[12]:

	customer_id	predicted_uplift	p_with_offer	p_without_offer	rank	decision
53999	492666	0.1098	0.2087	0.0989	54000	target
54000	379866	0.1098	0.2053	0.0954	54001	do not target
179999	519801	-0.9117	0.0489	0.9606	180000	do not target

12. Profile of customers with negative predicted uplift (Nice-to-have 3.2-5)

Customers whose predicted uplift is below zero are compared with the rest. Because the features are anonymised, differences are reported only as standardised mean differences, without business interpretation.

In [13]:
```
neg = u_final < 0
print(f"Share of test customers with negative predicted uplift: {neg.mean():.2%}")
yt, tt = y_te.values, t_te.values
obs_uplift = {}
for lbl, m in [("negative group", neg), ("rest", ~neg)]:
    obs_uplift[lbl] = yt[m & (tt == 1)].mean() - yt[m & (tt == 0)].mean()
    print(f"Observed uplift in {lbl:14s}: {obs_uplift[lbl]:+.4f} (n = {m.sum():,})")
Xn, Xr = X_te[neg], X_te[~neg]
smd_neg = ((Xn.mean() - Xr.mean()) / np.sqrt((Xn.var() + Xr.var()) / 2)).rename("SMD (negative vs rest)")
top_neg = smd_neg.reindex(smd_neg.abs().sort_values(ascending=False).index).head(8)
display(top_neg.to_frame().round(3))
```

Out[13]:
```
Share of test customers with negative predicted uplift: 35.15%
Observed uplift in negative group: -0.1146 (n = 63,279)
Observed uplift in rest          : +0.1385 (n = 116,721)
```

Out[13]:

	SMD (negative vs rest)
X_4	0.104
X_42	0.065
X_33	0.049
X_49	-0.040
X_38	0.033
X_17	-0.029
X_15	-0.027
X_48	-0.027

13. Feature influence on the uplift estimates (Nice-to-have 3.2-6)

Permutation importance on the Qini coefficient. On a 60,000-customer sample of the test set, each feature is shuffled in turn (two repeats) and the final model's uplift is re-predicted. The drop in the Qini coefficient measures how much the ranking quality depends on that feature. This uses the fitted model directly, so no surrogate approximation is needed.

In [14]:

```
rs = np.random.RandomState(SEED)
sample = rs.choice(len(X_te), 60000, replace=False)
Xs, ys_, ts_ = X_te.iloc[sample].reset_index(drop=True), y_te.iloc[sample].values, t_te.iloc[sample].values
model_final = FITTED[FINAL]
base_q = qini_auc_score(ys_, model_final.predict(Xs), ts_)
drops = {}
for f in FEATURES:
    d = []
    for r in range(2):
        Xp = Xs.copy()
        Xp[f] = np.random.RandomState(SEED + r).permutation(Xp[f].values)
        d.append(base_q - qini_auc_score(ys_, model_final.predict(Xp), ts_))
    drops[f] = np.mean(d)
imp = pd.Series(drops).sort_values(ascending=False)
print(f"Baseline Qini on the sample: {base_q:.4f}")
print(f"Features whose permutation lowers Qini by > 0.005: {int((imp > 0.005).sum())} of 50")
print(f"Top-10 features account for {imp.clip(lower=0).head(10).sum() / imp.clip(lower=0).sum():.1%} of the total positive
drop")
display(imp.head(10).to_frame("Qini drop").round(4))
imp.to_csv(RES_DIR / "feature_influence.csv", header=["qini_drop"])

fig, ax = plt.subplots(figsize=(6.4, 4.0))
top = imp.head(15)[::-1]
ax.barh(top.index, top.values, color="#eb6834", height=0.65)
ax.set_xlabel("Drop in Qini coefficient when the feature is shuffled")
ax.set_title(f"Top 15 features driving the {FINAL} ranking")
fig.tight_layout(); fig.savefig(FIG_DIR / "fig7_feature_influence.png"); plt.show()
```

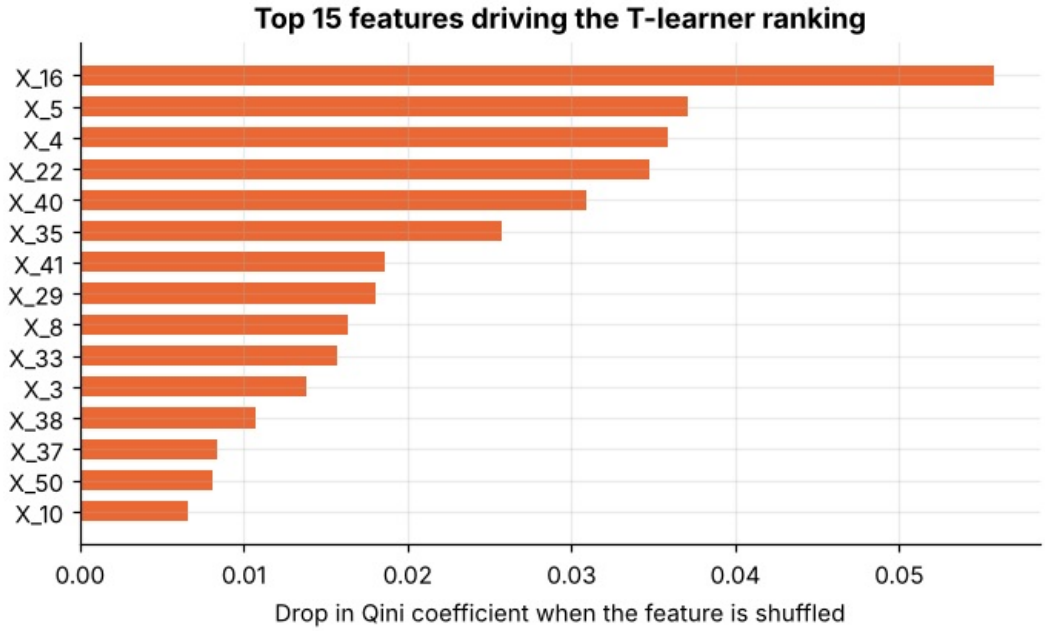
Out[14]:

```
Baseline Qini on the sample: 0.2365
Features whose permutation lowers Qini by > 0.005: 18 of 50
Top-10 features account for 73.9% of the total positive drop
```

Out[14]:

	Qini drop
X_16	0.0558
X_5	0.0371
X_4	0.0359
X_22	0.0348
X_40	0.0309
X_35	0.0257
X_41	0.0186
X_29	0.0181
X_8	0.0164
X_33	0.0157

Out[14]:



14. Success-criteria check (Proposal §4)

In [15]:
```
elapsed_min = (time.time() - T0) / 60
fin = summary.loc[FINAL]
top30_uplift = res.loc[FINAL, "Uplift@30%"]
crit = [
    ("1 Data size", f"{len(df):,} rows used, {len(idx_te):,} held out", len(df) >= 100_000),
    ("2 Group identification", f"max treatment-share deviation {100*max(abs(t_tr.mean()-t.mean()), abs(t_te.mean()-
t.mean())):.3f} pp",
     df["T"].isin([0, 1]).all() and max(abs(t_tr.mean()-t.mean()), abs(t_te.mean()-t.mean())) <= 0.01),
    ("3 Group balance", f"max |SMD| = {smd.abs().max():.4f}", smd.abs().max() < 0.1),
    ("4 Coverage", f"{np.isfinite(u_final).sum():,} of {len(u_final):,} scores finite", np.isfinite(u_final).all()),
    ("5 Better than random", f"Qini = {res.loc[FINAL, 'Qini coefficient']:.4f}", res.loc[FINAL, "Qini coefficient"] > 0),
    ("6 Better than conversion model", f"{res.loc[FINAL, 'Qini coefficient']:.4f} vs {res.loc['Conversion model', 'Qini
coefficient']:.4f}",
     res.loc[FINAL, "Qini coefficient"] > res.loc["Conversion model", "Qini coefficient"]),
    ("7 Stability", f"positive Qini in {int(fin['positive_splits'])} of 5 splits", fin["positive_splits"] >= 4),
    ("8 Targeting value", f"uplift@30% {top30_uplift:.4f} vs overall {overall_uplift_te:.4f}", top30_uplift >
overall_uplift_te),
    ("9 Reproducibility", f"end-to-end runtime {elapsed_min:.1f} min, fixed seeds", elapsed_min < 60),
]
crit_df = pd.DataFrame(crit, columns=["criterion", "measured value", "met"]).set_index("criterion")
display(crit_df)
json.dump({"final_model": FINAL, "best_params": {k: v for k, v in BEST.items()},
           "ate": ate, "ate_ci": [ate - 1.96*se, ate + 1.96*se], "overall_uplift_test": overall_uplift_te,
           "max_smd": float(smd.abs().max()), "neg_share": float(neg.mean()), "perm_base_qini": base_q, "obs_uplift_neg":
obs_uplift["negative group"], "obs_uplift_rest": obs_uplift["rest"],
           "top_smd_neg": {k: float(v) for k, v in top_neg.items()},
           "runtime_min": elapsed_min}, open(RES_DIR / "notebook_summary.json", "w"), indent=2, default=float)
print(f"All criteria met: {bool(crit_df['met'].all())}")
print(f"Total runtime: {elapsed_min:.1f} minutes")
```

Out[15]:

criterion	measured value	met
1 Data size	600,000 rows used, 180,000 held out	True
2 Group identification	max treatment-share deviation 0.000 pp	True
3 Group balance	max |SMD| = 0.0046	True
4 Coverage	180,000 of 180,000 scores finite	True
5 Better than random	Qini = 0.2450	True
6 Better than conversion model	0.2450 vs 0.0965	True
7 Stability	positive Qini in 5 of 5 splits	True
8 Targeting value	uplift@30% 0.2244 vs overall 0.0495	True
9 Reproducibility	end-to-end runtime 16.1 min, fixed seeds	True

Out[15]:
```
All criteria met: True
Total runtime: 16.1 minutes
```

Uplift Modeling for Personalized Marketing in Telecommunications: Ranking Subscribers by the Incremental Effect of a Campaign

Abilkhay Askar and Kolzhabay Olzhas

Group Big Data Analysis 2406

Astana IT University

Applied Machine Learning

October 30, 2026

Abstract

Telecommunications operators usually target marketing offers at the customers who are most likely to convert, yet many of those customers would convert anyway, and some react negatively to being contacted. This study asks which subscribers should receive an offer so that the number of additional (incremental) conversions is maximized. We used the synthetic MegaFon Uplift Competition dataset, a randomized experiment with 600,000 subscribers, 50 anonymized features, a treatment flag and a binary conversion outcome. After confirming that treatment and control groups were balanced (maximum standardized mean difference 0.0046), we compared three uplift approaches built on histogram gradient-boosted trees, namely the S-learner, the T-learner and class transformation, with a linear T-learner, a conventional conversion classifier and random targeting. Base-learner settings were tuned by cross-validated Qini coefficient, and all models were evaluated on a 180,000-customer held-out test set and on five repeated splits. The T-learner performed best, with a test Qini coefficient of 0.245 (mean 0.244 over five splits) against 0.096 for the conversion classifier and approximately zero for random targeting. Contacting the top 30% of customers ranked by the T-learner produced an observed uplift of 22.4%, compared with 5.0% for the whole test set, which corresponds to about 4.5 times as many incremental conversions as random targeting. The results show that uplift models, not response models, should drive campaign targeting when the goal is incremental impact.

Keywords: uplift modeling, heterogeneous treatment effects, meta-learners, Qini coefficient, telecommunications marketing

Uplift Modeling for Personalized Marketing in Telecommunications

Introduction

Mobile operators run frequent marketing campaigns, such as discounted tariff upgrades, bundled data packages or loyalty bonuses. Each contact has a cost, and an operator with millions of subscribers can afford to reach only part of the base. The common way to choose whom to contact is to train a response (propensity) model that predicts the probability of conversion and to send the offer to the customers with the highest scores. This answers the question “Who is likely to convert?”, which is not the question the business needs to answer. A customer with a high propensity score may convert whether or not the offer arrives, so the campaign spends money without changing the outcome, and some customers are less likely to convert when contacted (Radcliffe, 2007; Ascarza, 2018).

Uplift modeling replaces the question with “Who is more likely to convert *because* of the offer?” In the potential-outcomes framework (Rubin, 1974), each customer with feature vector x has two outcomes: $Y(1)$ if treated and $Y(0)$ if not. The quantity of interest is the conditional average treatment effect, $\text{uplift}(x) = \mathrm{E}[Y(1) - Y(0) \mid X = x]$. Only one potential outcome is ever observed, so uplift cannot be used as a training label. When treatment is assigned at random, however, $\text{uplift}(x)$ equals $\mathrm{P}(Y = 1 \mid x, T = 1) - \mathrm{P}(Y = 1 \mid x, T = 0)$ and can be estimated from the treatment and control groups (Gutierrez & Gérardy, 2017).

This distinction matters for telecommunications in particular. Operators run randomized holdout groups routinely, they have rich usage data for every subscriber, and their campaigns are large enough for small improvements in targeting to translate into thousands of extra conversions. Typical applications include selecting customers for upsell offers, deciding who

should receive retention incentives, and avoiding “sleeping dogs”, customers whom a contact pushes away.

The research question of this project is therefore: *Which subscribers should receive a marketing offer in order to maximize incremental conversions, and does an uplift model rank subscribers better than random targeting and better than a conventional conversion model?* The objectives are to (a) check that the experimental data support causal estimation, (b) estimate individual uplift with at least two meta-learning approaches, (c) evaluate them with uplift-specific metrics on held-out data and across repeated splits, and (d) turn the best model into a ranked targeting list for a fixed contact budget.

Literature Review

Early uplift work came from direct marketing. Radcliffe (2007) argued that campaigns should be targeted on predicted lift rather than predicted response and introduced the Qini curve and Qini coefficient, which measure how many incremental responses a ranking captures compared with random selection. Radcliffe and Surry (2011) later proposed significance-based uplift trees that split on the difference in treatment effect between child nodes. Rzepakowski and Jaroszewicz (2012) built decision trees whose splitting criteria are divergences between the treatment and control outcome distributions, and Athey and Imbens (2016) developed honest recursive partitioning with valid confidence intervals for heterogeneous effects.

A second line of work reuses ordinary supervised learners. Künzel et al. (2019) formalized these *meta-learners*. The S-learner fits one model with the treatment flag as an extra feature, the T-learner fits separate models for treated and control units, and the X-learner combines the two with imputed effects. Jaskowski and Jaroszewicz (2012) proposed the class-transformation approach, which turns uplift estimation into a single binary classification problem

when treatment is assigned with probability one half. Gutierrez and Gérardy (2017) and Devriendt et al. (2018) reviewed these families and concluded that no single method dominates; performance depends on the data, the strength of the treatment effect and the base learner. Ensemble learners such as gradient-boosted trees (Friedman, 2001; Ke et al., 2017) are attractive base models because they capture non-linear interactions between features and treatment.

Evaluation is a recognized difficulty because the true individual effect is never observed. The standard remedy is to rank customers and compare the observed treatment and control response rates within ranked groups, summarized by Qini and uplift curves (Radcliffe, 2007; Gutierrez & Gérardy, 2017). Large public benchmarks, such as the Criteo advertising dataset (Diemert et al., 2018) and the MegaFon Uplift Competition (MegaFon, 2021), have made such comparisons reproducible. In marketing research, Ascarza (2018) showed with field experiments that targeting the customers with the highest churn risk can be less effective than targeting those with the highest predicted response to the intervention.

Three gaps motivate this project. First, many applied studies still select and tune models by classification accuracy or ROC AUC, although these metrics say nothing about how well a model ranks customers by treatment effect. Second, published comparisons often report a single train/test split, which hides how stable the ranking is. Third, few studies translate uplift scores into an explicit budgeted targeting decision and compare it with the response model that practitioners actually use. This project addresses all three on a large telecommunications dataset.

Data

Source and Structure

The study uses the MegaFon Uplift Competition dataset (MegaFon, 2021), released by the Russian operator MegaFon on the Open Data Science platform and distributed through the

scikit-uplift library (Shevchenko, n.d.). The data are synthetic: they were generated to imitate a telecommunications marketing experiment, so no real customer records are involved. The labelled file contains 600,000 rows. Each row has a treatment flag (“treatment” or “control”), 50 anonymized numeric features *X*_1 to *X*_50 whose business meaning is not published, and a binary conversion outcome. The distributed file contains no explicit subscriber identifier, so the row index was used as the customer id. A separate 900,000-row competition test file has no labels and was not used.

Table 1

Summary of the MegaFon Uplift Dataset

Property	Value
Labelled observations	600,000
Features	50 numeric (X_1 … X_50), anonymized
Treatment group	300,368 (50.06%), conversion rate 22.89%
Control group	299,632 (49.94%), conversion rate 17.94%
Average treatment effect	0.0495 (95% CI 0.0475 to 0.0516)
Missing values / duplicate rows	0 / 0
Maximum |SMD| between groups	0.0046 (threshold 0.1)

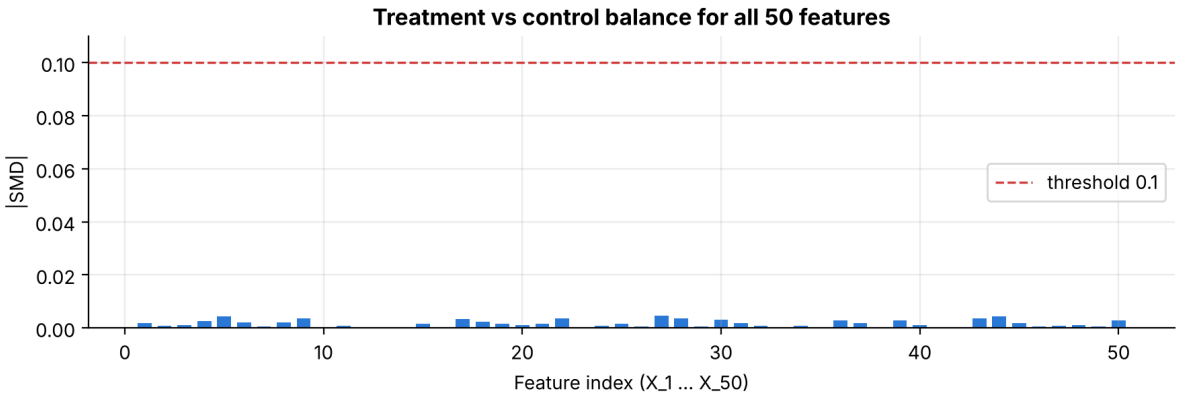
Note. SMD = standardized mean difference. CI = confidence interval. Computed in the project notebook.

Validation and Balance

The data were checked for missing values, duplicate records and column types; none of the 50 features contained missing values and no row was duplicated. The offer raised the conversion rate from 17.94% to 22.89%, an average treatment effect of 4.95 percentage points (a relative lift of 27.6%). Because uplift estimation relies on random assignment, the distribution of every feature was compared between the two groups with the standardized mean difference SMD = (mean_T − mean_C) / √((var_T + var_C)/2). All 50 values were far below the common threshold of 0.1 (largest 0.0046), which is consistent with randomized assignment (Figure 1).

Figure 1

Absolute Standardized Mean Difference Between Treatment and Control for the 50 Features



Note. The dashed line marks the 0.1 balance threshold. All features are below 0.005.

Preprocessing and Train/Test Split

No imputation was needed. Features were left unscaled for the tree-based learners, which are invariant to monotone transformations, and were standardized inside a pipeline for the logistic-regression baseline. The data were split into 70% training (420,000 rows) and 30% test (180,000 rows), stratified on the four combinations of treatment and conversion. The treatment share and conversion rate were identical to four decimals in the full, training and test sets. The test set was used only for final evaluation; hyper-parameter tuning used cross-validation inside the training set.

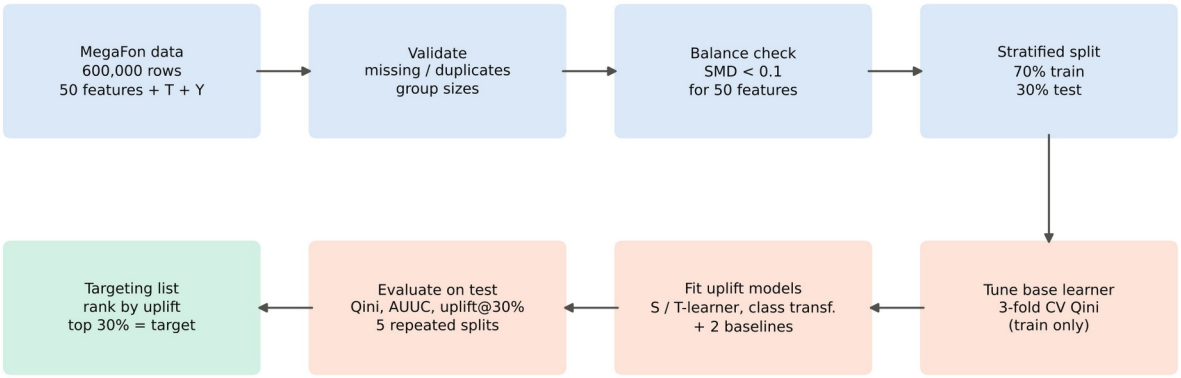
Methodology

Pipeline

Figure 2 shows the full pipeline, from loading the data to producing the targeting list. All steps run in one Python notebook with fixed random seeds.

Figure 2

Project Pipeline From Raw Data to Targeting Decision

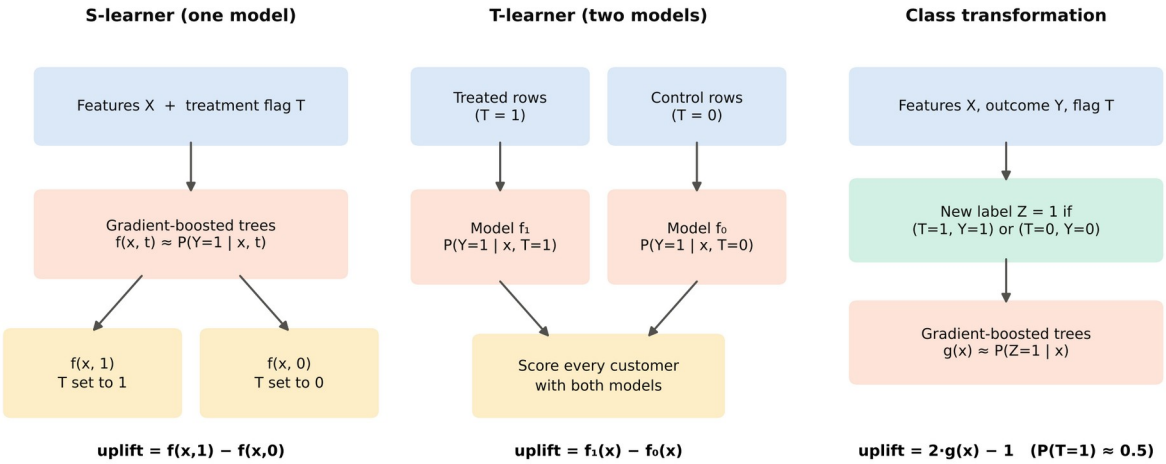


Uplift Models

Three uplift approaches were implemented with the scikit-uplift wrappers (Figure 3). The **S-learner** (SoloModel) trains a single classifier on [X, T] and estimates uplift as f(x, 1) − f(x, 0). The **T-learner** (TwoModels, vanilla mode) trains one classifier on treated customers and another on control customers and subtracts their predicted probabilities. **Class transformation** creates the label $Z = 1$ when ($T = 1$, $Y = 1$) or ($T = 0$, $Y = 0$); with a treatment probability of about 0.5, uplift(x) = 2P($Z = 1 \mid x$) − 1 (Jaskowski & Jaroszewicz, 2012). A T-learner with standardized logistic regression was added as a linear reference.

Figure 3

Architectures of the Three Uplift Meta-Learners



Two baselines represent current practice. **Random targeting** assigns each customer a random score. The **conversion model** is the same gradient-boosted classifier trained on *X* only (all training customers) and ranks customers by predicted conversion probability, which is how a response model is used in campaign selection.

Base Learner, Parameters and Training Procedure

All tree-based models use scikit-learn's HistGradientBoostingClassifier (Pedregosa et al., 2011), a histogram-based implementation of gradient-boosted decision trees (Friedman, 2001; Ke et al., 2017). Each model is an additive ensemble of shallow regression trees fitted sequentially to the gradient of the binary log-loss; tree models have no layers, activation functions, batches or epochs in the neural-network sense. Fixed settings were min_samples_leaf = 100, L2 regularization = 1.0, no early stopping and random_state = 42. The learning rate, the maximum number of leaves per tree and the number of boosting iterations were tuned over four configurations by the mean Qini coefficient in 3-fold stratified cross-validation on the training set (Table 2), as planned in the proposal, instead of by accuracy. The logistic-regression baseline used the L-BFGS optimizer with up to 1,000 iterations.

Table 2

Cross-Validated Qini Coefficient for Each Base-Learner Configuration

Model	Learning rate	Max leaves	Iterations	CV Qini M (SD)
S-learner	0.10	31	200	0.2313 (0.0012)
S-learner	0.05	31	300	0.2310 (0.0015)
S-learner	0.10	15	200	0.2179 (0.0020)
S-learner	0.10	63	150	0.2365 (0.0007)
T-learner	0.10	31	200	0.2364 (0.0015)
T-learner	0.05	31	300	0.2361 (0.0024)
T-learner	0.10	15	200	0.2292 (0.0015)
T-learner	0.10	63	150	0.2380 (0.0003)
Class transformation	0.10	31	200	0.2184 (0.0025)
Class transformation	0.05	31	300	0.2233 (0.0033)
Class transformation	0.10	15	200	0.2177 (0.0021)
Class transformation	0.10	63	150	0.2179 (0.0022)

Note. 3-fold stratified cross-validation on the 420,000 training rows. Selected configurations: 0.10/63/150 for the S- and T-learner, 0.05/31/300 for class transformation. The conversion model uses the S-learner setting.

Evaluation Metrics

Accuracy and ROC AUC were deliberately not used for model selection, because a model can predict conversion well and still rank customers poorly by treatment effect (Radcliffe, 2007). Three uplift metrics from scikit-uplift were computed on the test set. The **Qini curve** plots the cumulative number of incremental conversions, $Y^{T}(k) - Y^{C}(k) \cdot N^{T}(k)/N^{C}(k)$, when the top k customers by score are contacted. The **Qini coefficient** is the area between this curve and the random-targeting line, normalized by the area of a perfect model, so random targeting scores 0 and a perfect ranking scores 1; it is also the official metric of the MegaFon competition. **AUUC** applies the same normalization to the uplift curve. **Uplift@30%** is the treatment conversion rate minus the control conversion rate among the 30% of customers with the highest scores, which matches the planned contact budget. Stability was measured by retraining every approach on five

stratified 70/30 splits (seeds 42, 7, 123, 2024 and 99), and the final model was the approach with the highest mean Qini coefficient over these splits.

Experiments and Results

Model Comparison on the Test Set

Table 3

Uplift Metrics on the 180,000-Customer Test Set and Across Five Repeated Splits

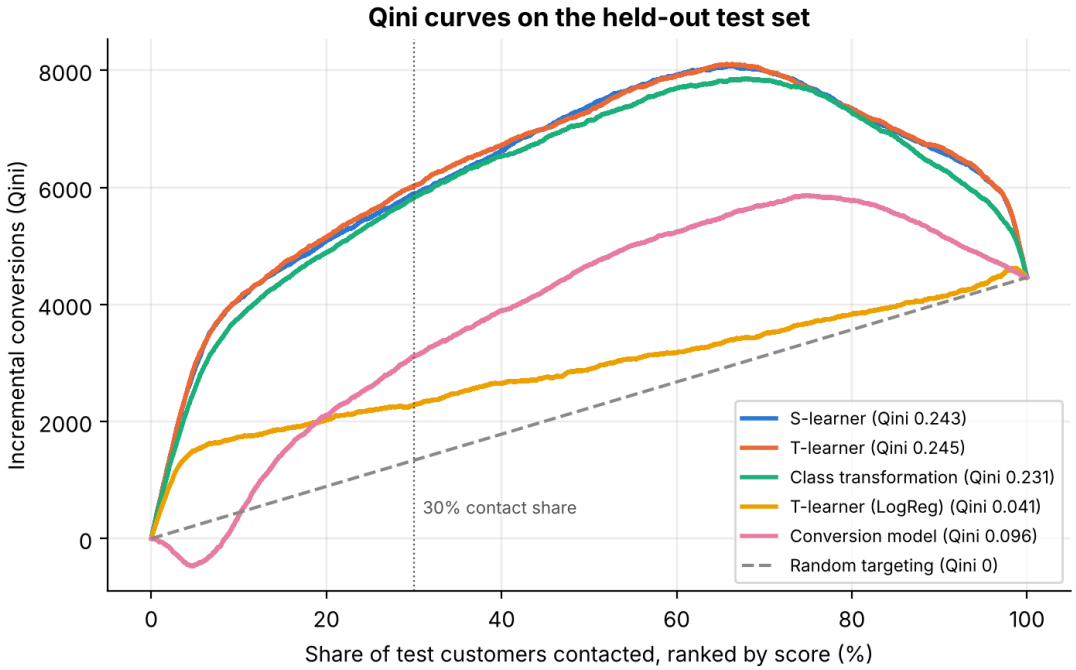
Model	Qini	AUUC	Uplift@30%	Qini, 5 splits M (SD)	Splits Qini > 0
T-learner	0.2450	0.1695	0.2244	0.2438 (0.0026)	5/5
S-learner	0.2430	0.1679	0.2193	0.2407 (0.0046)	5/5
Class transformation	0.2315	0.1599	0.2165	0.2304 (0.0027)	5/5
Conversion model	0.0965	0.0666	0.1153	0.0958 (0.0026)	5/5
T-learner (LogReg)	0.0405	0.0282	0.0852	0.0356 (0.0030)	5/5
Random	-0.0033	-0.0023	0.0454	-0.0021 (0.0034)	1/5

Note. Observed uplift on the whole test set = 0.0495. Qini and AUUC are normalized (random = 0, perfect = 1). Higher is better for all columns.

Table 3 summarizes the comparison. The three gradient-boosted uplift models were close to each other and far ahead of both baselines. The T-learner achieved the highest test Qini coefficient (0.245), followed by the S-learner (0.243) and class transformation (0.231). The conversion model, which is what a response-based campaign would use, reached only 0.096, and random targeting scored -0.003, close to zero as expected. The ordering was the same on all five repeated splits: the T-learner had the highest mean Qini (0.244) and the smallest standard deviation (0.0026), so it was selected as the final model. Figure 4 shows the Qini curves.

Figure 4

Qini Curves of All Models on the Held-Out Test Set

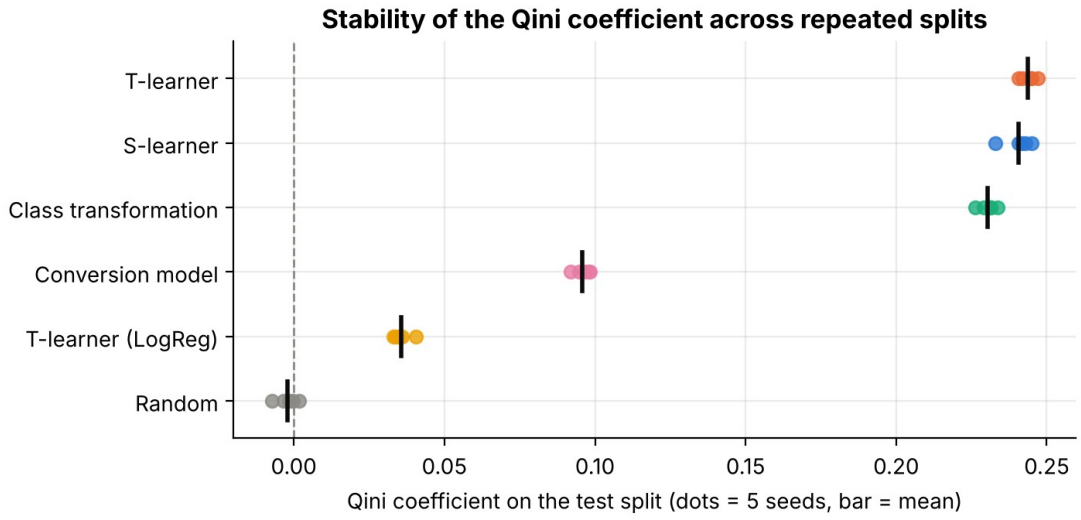


Note. Customers are contacted in order of decreasing score. The dashed line is random targeting; the dotted vertical line marks a 30% contact share.

The curves of the three boosted uplift models rise steeply over the first 10% to 20% of customers and then flatten. Beyond roughly 70% they decline, because the lowest-ranked customers convert *less* often when treated, so contacting them removes conversions. The conversion-model curve stays well below the uplift models: customers who are very likely to convert are not, in general, the ones whom the offer persuades.

Figure 5

Qini Coefficient on Five Repeated Train/Test Splits



Note. Each dot is one split; the black bar is the mean over the five splits.

Why Some Models Performed Better

The T-learner's small advantage over the S-learner is consistent with the structure of the data. Because treatment and control groups are equally large (about 210,000 training customers each), each of the two models has ample data, and a separate model for each arm can represent completely different response surfaces. The S-learner must learn the treatment effect through splits on a single binary feature, and gradient boosting may use that feature in only part of its trees, which tends to shrink estimated effects toward zero (Künzel et al., 2019). Class transformation optimizes a related but noisier target: the transformed label mixes treatment and outcome information, and its variance grows with the base conversion rate, which explains its slightly lower Qini. Tuning confirmed that more complex trees (63 leaves) helped both the S- and T-learner, while class transformation preferred a lower learning rate and more iterations.

The linear T-learner reached a Qini coefficient of only 0.041. Its weakness shows that the treatment effect in this dataset depends on non-linear combinations of features that a linear

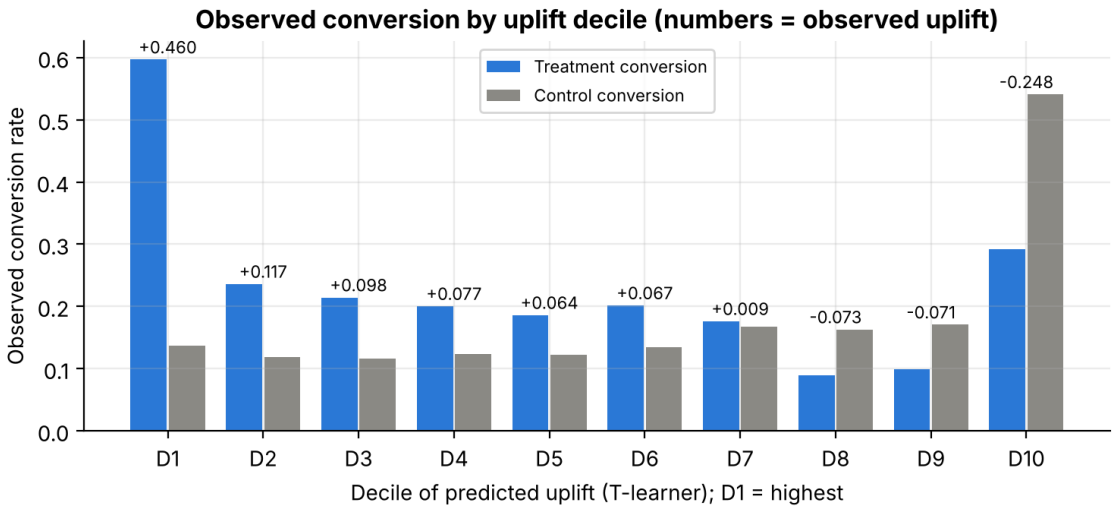
decision boundary cannot represent. The conversion model ranked customers better than random but far worse than any boosted uplift model, which directly confirms the motivating argument of the project.

Uplift by Decile

Figure 6 and Table 4 show the observed treatment and control conversion rates within each decile of the T-learner's predicted uplift. Observed uplift falls almost monotonically from +0.460 in the top decile to -0.248 in the bottom decile. In the first decile, the offer raised conversion from 13.7% to 59.7%. In the last decile the pattern reversed: control customers converted at 54.1% but treated customers only at 29.3%. These are the customers that the uplift literature calls sleeping dogs; a response model would rank many of them highly because their baseline conversion rate is high.

Figure 6

Observed Conversion Rates by Decile of Predicted Uplift (T-Learner)



Note. D1 contains the 10% of test customers with the highest predicted uplift. Numbers above the bars are the observed uplift (treatment rate minus control rate).

Table 4

Observed Response Rates by Decile of T-Learner Predicted Uplift

Decile	n treatment	n control	Treatment rate	Control rate	Observed uplift
D1	8917	9083	59.7%	13.7%	+0.460
D2	8946	9054	23.6%	11.9%	+0.117
D3	8994	9006	21.4%	11.7%	+0.098
D4	9059	8941	20.1%	12.4%	+0.077
D5	8961	9039	18.6%	12.2%	+0.064
D6	9130	8870	20.2%	13.5%	+0.067
D7	9027	8973	17.6%	16.7%	+0.009
D8	9038	8962	9.0%	16.3%	-0.073
D9	9019	8981	9.9%	17.1%	-0.071
D10	9019	8981	29.3%	54.1%	-0.248

Campaign Efficiency and Targeting Decision

Uplift scores were turned into a budgeted decision. For a contact share *k*, the expected number of incremental conversions among the selected customers was estimated as the number contacted multiplied by their observed uplift; random targeting of the same number of customers yields that number multiplied by the overall test uplift (0.0495). At the planned 30% budget (54,000 of 180,000 test customers), the T-learner selection is associated with about 12,119 incremental conversions, compared with 6,224 for the conversion model and 2,674 for random targeting, a 4.5-fold gain (Table 5, Figure 7). The relative gain is largest for small budgets, because the most persuadable customers are concentrated at the top of the ranking.

Table 5

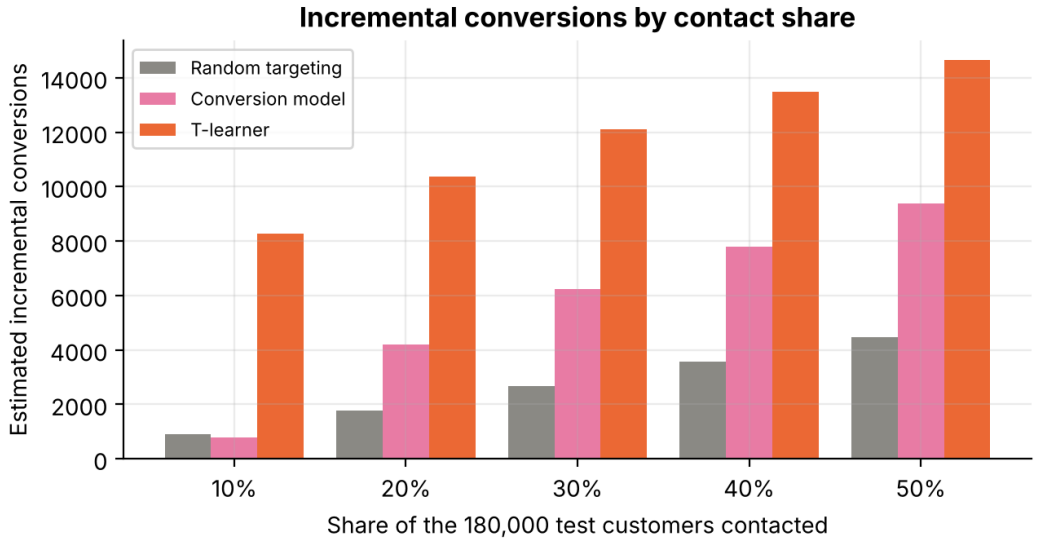
Estimated Incremental Conversions by Contact Share on the Test Set

Contact share	Customers	Random	Conversion model	T-learner	Gain vs. random
10%	18,000	891	783	8,280	9.3×
20%	36,000	1,783	4,204	10,379	5.8×
30%	54,000	2,674	6,224	12,119	4.5×
40%	72,000	3,566	7,799	13,477	3.8×
50%	90,000	4,457	9,372	14,639	3.3×

Note. Incremental conversions = customers contacted × observed uplift in the selected group.

Figure 7

Estimated Incremental Conversions by Contact Share



The final output of the system is a list of all 180,000 test customers sorted by predicted uplift, with the top 54,000 marked *target*. For each customer the list also gives the two potential-outcome probabilities. For example, the highest-ranked customer has an estimated conversion probability of 0.90 with the offer and 0.07 without it, whereas the lowest-ranked customer has 0.05 with the offer and 0.96 without it, so contacting them would be expected to cost a conversion.

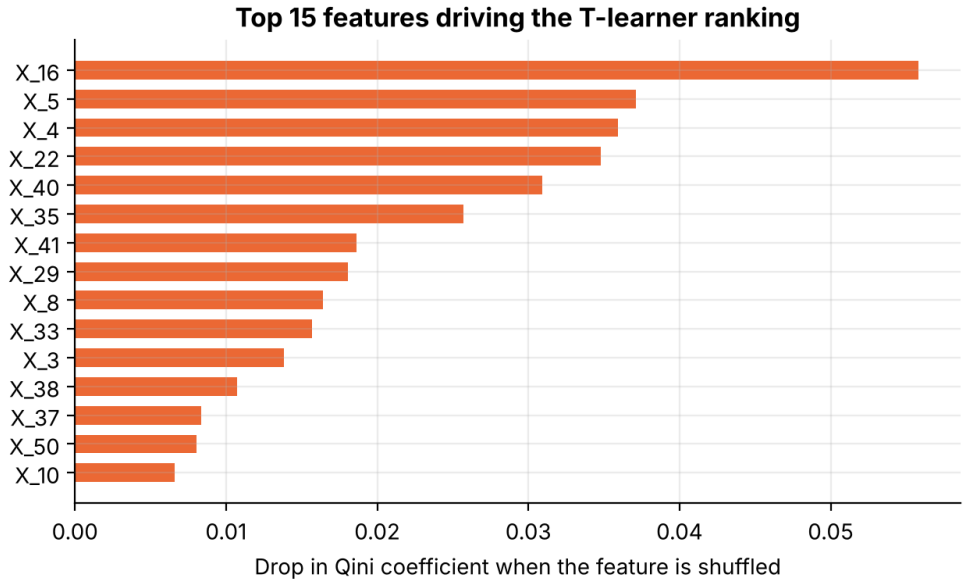
Negative-Uplift Customers and Feature Influence

The T-learner predicted negative uplift for 35.2% of test customers. The prediction is supported by the observed data: in this group the treated conversion rate was 11.5 percentage points *lower* than the control rate, whereas in the remaining customers it was 13.8 points higher. The features that differ most between the two groups were X_4 (+0.104), X_42 (+0.065), X_33 (+0.049) in standardized mean difference. Because the features are anonymized, these differences are reported without business interpretation.

Feature influence was measured by permutation importance (Breiman, 2001) on the Qini coefficient: each feature was shuffled in a 60,000-customer test sample and the drop in the T-learner's Qini was recorded. Only 18 of the 50 features reduced the Qini by more than 0.005 when shuffled; the most influential were X_16, X_5, X_4, X_22 and X_40 (Figure 8). The treatment effect is therefore driven by a small subset of the anonymized features, while most features contribute little to the ranking.

Figure 8

Permutation Importance of the 15 Most Influential Features for the T-Learner Ranking



Note. Importance is the mean decrease in the Qini coefficient over two random permutations of each feature.

Success Criteria

Table 6

Proposal Success Criteria and Measured Results

Criterion	Measured value	Status
1 Data size	600,000 rows used, 180,000 held out	Met
2 Group identification	100% assigned; share deviation < 0.01 pp	Met
3 Group balance	max |SMD| = 0.0046 < 0.1	Met
4 Coverage	180,000 of 180,000 scores finite	Met
5 Better than random	Qini = 0.2450 > 0	Met
6 Better than conversion model	0.2450 vs. 0.0965	Met
7 Stability	Qini > 0 in 5 of 5 splits	Met
8 Targeting value	uplift@30% 0.2244 vs. 0.0495 overall	Met
9 Reproducibility	14 min end to end, fixed seeds	Met

All nine acceptance criteria defined in the proposal were met (Table 6), and all six optional requirements were completed: a third approach (class transformation), tuning by cross-validated Qini, AUUC and decile reporting, campaign-efficiency estimates for 10% to 50% contact shares, a profile of negative-uplift customers, and feature influence.

Challenges and Limitations

Several limitations qualify the results. The dataset is synthetic and its features are anonymized, so the absolute size of the effects (for example, the very large uplift in the top decile) may be stronger than in real campaigns, and the findings cannot be turned into business rules about specific customer attributes. Overfitting was controlled by tuning on training folds only, by the minimum leaf size and L2 penalty, and by checking stability on five independent splits, where the standard deviation of the Qini coefficient stayed below 0.005 for every boosted model; underfitting was evident only for the linear model. The campaign-efficiency figures use observed rates within the selected group and assume that costs per contact are equal and that no other campaign interferes. Finally, the analysis considers one treatment and one binary outcome; it does not model revenue, contact cost or multiple offers.

Future Work

Natural extensions are the X-learner and R-learner (Künzel et al., 2019), uplift random forests or causal forests (Athey & Imbens, 2016), cost-sensitive targeting that maximizes profit rather than conversions, multi-treatment models for choosing between several offers, and validation on a real operator's randomized campaign, ideally with a prospective test of the targeting list against random and response-based selection.

Conclusion

This project built and evaluated an uplift-modeling system that decides which telecommunications subscribers should receive a marketing offer. On the 600,000-customer MegaFon experiment, gradient-boosted meta-learners ranked customers by incremental effect far better than both random targeting and a conventional conversion model. The T-learner was the most accurate and the most stable approach, with a test Qini coefficient of 0.245 against 0.096 for the conversion model, and contacting its top 30% of customers produced an observed uplift of 22.4%, compared with 5.0% on average.

The practical implication is that campaign targeting should be based on predicted uplift, not predicted response. A response model wastes contacts on customers who would convert anyway and can even select customers whom the offer discourages; about 35.2% of customers in this dataset had negative predicted uplift. With a randomized holdout group, which most operators already maintain, the same pipeline can be applied to upsell, cross-sell and retention campaigns to raise incremental conversions for a fixed budget and to avoid contacting customers who react badly.

References

Ascarza, E. (2018). Retention futility: Targeting high-risk customers might be ineffective. *Journal of Marketing Research, 55*(1), 80–98. https://doi.org/10.1509/jmr.16.0163

Athey, S., & Imbens, G. (2016). Recursive partitioning for heterogeneous causal effects. *Proceedings of the National Academy of Sciences, 113*(27), 7353–7360. https://doi.org/10.1073/pnas.1510489113

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Devriendt, F., Moldovan, D., & Verbeke, W. (2018). A literature survey and experimental evaluation of the state-of-the-art in uplift modeling: A stepping stone toward the development of prescriptive analytics. *Big Data, 6*(1), 13–41. https://doi.org/10.1089/big.2017.0104

Diemert, E., Betlei, A., Renaudin, C., & Amini, M.-R. (2018). A large scale benchmark for uplift modeling. In *Proceedings of the AdKDD and TargetAd Workshop, KDD 2018*. ACM.

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics, 29*(5), 1189–1232. https://doi.org/10.1214/aos/1013203451

Gutierrez, P., & Gérardy, J.-Y. (2017). Causal inference and uplift modelling: A review of the literature. In *Proceedings of the 3rd International Conference on Predictive Applications and APIs* (Proceedings of Machine Learning Research, Vol. 67, pp. 1–13). PMLR.

Jaskowski, M., & Jaroszewicz, S. (2012). Uplift modeling for clinical trial data. In *ICML 2012 Workshop on Clinical Data Analysis*.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30* (pp. 3146–3154).

Künzel, S. R., Sekhon, J. S., Bickel, P. J., & Yu, B. (2019). Metalearners for estimating heterogeneous treatment effects using machine learning. *Proceedings of the National Academy of Sciences, 116*(10), 4156–4165. https://doi.org/10.1073/pnas.1804597116

MegaFon. (2021). *MegaFon Uplift Competition: Data* [Data set]. Open Data Science (ODS.ai). https://ods.ai/tracks/df21-megafon/competitions/megafon-df21-comp/data

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Radcliffe, N. J. (2007). Using control groups to target on predicted lift: Building and assessing uplift models. *Direct Marketing Analytics Journal*, 14–21.

Radcliffe, N. J., & Surry, P. D. (2011). *Real-world uplift modelling with significance-based uplift trees* (Technical Report TR-2011-1). Stochastic Solutions.

Rubin, D. B. (1974). Estimating causal effects of treatments in randomized and nonrandomized studies. *Journal of Educational Psychology, 66*(5), 688–701. https://doi.org/10.1037/h0037350

Rzepakowski, P., & Jaroszewicz, S. (2012). Decision trees for uplift modeling with single and multiple treatments. *Knowledge and Information Systems, 32*(2), 303–327. https://doi.org/10.1007/s10115-011-0434-0

Shevchenko, M. (n.d.). *scikit-uplift: Uplift modeling in scikit-learn style in Python* (Version 0.5.1) [Computer software]. https://www.uplift-modeling.com